

Guidance in Diffusion Models for Target-Aware Molecular Generation

by

Jeremy Theodore Zay
Class of 2026

A thesis submitted to the
faculty of Wesleyan University
in partial fulfillment of the requirements for the
Degree of Bachelor of Arts
with Departmental Honors in Computer Science and College of
Integrative Sciences

Guidance in Diffusion Models for Target-Aware Molecular Generation

Abstract

This thesis investigates whether the sampling of a target-aware molecular diffusion model can be improved by applying guidance from an external scoring function. Using TargetDiff [1] on mutant p53 Y220C [2], we train a surrogate affinity model from docked generated molecules [3] and test whether it can guide reverse diffusion toward samples with higher binding affinity.

Although the surrogate is able to predict affinity well on clean molecules, the improvement it has on generation is limited. To better understand this behavior, we introduce PolyGen2D, a controlled synthetic testbed for studying guidance in a simpler coordinate-based diffusion setting. The results suggest that guidance is useful when the guidance model is compatible with the noisy intermediate states encountered during reverse diffusion.

Acknowledgements

There are a few important people I'd like to acknowledge.

Firstly, I'd like to thank my PI, Professor Kelly M. Thayer, for your endless guidance (no pun intended) and support during the process of making this.

Thank you very much to Professor Manfredi and Professor Krizanc for agreeing to read this.

Thank you to Mom and Dad for attempting to mitigate my procrastination efforts (it's hopeless), and for always supporting me and believing in me.

Thank you to Grandpa, for always understanding.

Thank you to Yehor, for having as much fascination in machine learning as I do.

Thank you to Ambrose, for enjoying so many lab meetings together with me. I'm grateful to have found the perfect friend.

And thanks to you, the reader, for enduring through this thesis!

“What I cannot create, I do not understand.”

— Richard Feynman

Contents

List of Figures	ix
List of Tables	x
1 Introduction	1
2 Background	3
2.1 Notation	3
2.2 Drug Discovery Context	4
2.3 p53 and the Y220C Mutation	5
2.4 Molecular Docking	6
2.5 Molecular Representation	6
2.6 Deep Neural Networks	7
2.7 Graph Neural Networks	9
2.8 SE(3)-Equivariance	10
2.9 Pre-Diffusion Approaches	12
2.10 Diffusion Models	13

2.11	Denoising Diffusion Probabilistic Models	13
2.12	Guidance in Diffusion Models	15
3	Methods	17
3.1	Overview	17
3.2	TargetDiff Pipeline	18
3.2.1	Model Setup	18
3.2.2	Candidate Generation and Evaluation	19
3.3	Surrogate Guidance (Molecular)	19
3.3.1	Dataset Construction	19
3.3.2	Surrogate Model	20
3.3.3	Guidance Mechanism	20
3.4	PolyGen2D Testbed	22
3.4.1	Motivation	22
3.4.2	Data and Representation	23
3.4.3	Diffusion Models	23
3.4.4	Guidance Objectives	24
3.4.5	Surrogate Variants	25

3.5	Experimental Design	26
3.5.1	TargetDiff Experiments	26
3.5.2	PolyGen2D Experiments	26
3.5.3	Evaluation Metrics	27
4	Results	28
4.1	Baseline TargetDiff	28
4.2	Surrogate Predictive Performance	30
4.3	Guided PolyGen2D Performance	31
4.4	PolyGen2D Baseline Behavior	32
4.5	Guidance in PolyGen2D	33
4.6	Surrogate Variants	33
4.7	Architectural Results	34
5	Discussion	36
5.1	Core Finding	36
5.2	Why Molecular Gains Remain Limited	36
5.3	What PolyGen2D Clarifies	38

5.4 Architectural Considerations	38
5.5 Implications for Molecular Guidance	39
6 Conclusion	41
6.1 What This Thesis Establishes	41
6.2 What Remains Open	41
Appendix A: Mathematical Details	42
Appendix B: PolyGen2D Output	56
Appendix C: PolyGen2D Output	59
Appendix C: Scripts	62
References	71

List of Figures

2.1	Molecule represented as a graph.	7
3.1	TargetDiff Diffusion Process [1]	18
3.2	PolyGen2D-to-TargetDiff research workflow.	22
3.3	One guided reverse step in PolyGen2D	24
3.4	PolyGen2D methodology	27
4.1	Docked generated molecule on Y220C	28
4.2	Score distribution of generated molecules	29
4.3	Top generated candidates	29
4.4	Surrogate affinity model standalone performance.	30
4.5	PolyGen2D unguided fixed-size results across architectures.	32
4.6	PolyGen2D unguided vs. guided (fixed-sized MLP)	33
4.7	PolyGen2D Guided Sampling Performance	34
5.1	Overlay of guided and unguided generated molecules.	37
6.1	Trajectory of denoising a polygon.	56
6.2	PolyGen2D training data.	57
6.3	PolyGen2D generated data.	58
6.4	Molecules generated by TargetDiff.	59
6.5	More molecules generated by TargetDiff.	60
6.6	More overlayed guided and unguided molecules.	61

List of Tables

2.1	Main notation used.	3
4.1	Guided minus baseline.	31

1 Introduction

Eventually, the entire universe will be pure noise, as the last stars and black holes dissolve and the average temperature of the universe drifts towards absolute zero. Physics tells us that entropy can never decrease over time in an isolated system, so how is it possible that a machine learning model can learn to reverse chaos by removing the noise from a digital object? It seems paradoxical – the idea that an image made entirely of TV static can be manipulated into an indistinguishable photograph, or that a vector initialized from a normal distribution can be formed into a molecule that restores a mutant protein and cures a type of cancer.

Popularized by DALL-E and Stable Diffusion in image generation, current state-of-the-art diffusion models can generate realistic data with astounding accuracy, to the point where a generated image of a face can fool a human brain which has evolved over hundreds of thousands of years to recognize its most minute details.

A natural domain for the use of this technology is drug discovery, and this method has been adapted to enable generation of molecules that bind to a specific protein. The next phase of drug discovery – identifying a molecule that not only binds to but also restores the function of a mutant protein – is still an open frontier of research. As we pursue a computationally efficient way to score a molecule on the restorative effect it will have on a protein, we must have a framework prepared for a scoring function to be integrated into the molecular generation process once such a function

inevitably exists. In this thesis, we explore the application of guidance in diffusion models used for molecular generation. We analyze the failure modes of using a surrogate model to guide a diffusion model to generate molecules with stronger binding affinity, and we build PolyGen2D, a controlled test environment where these modes can be isolated.

2 Background

2.1 Notation

Symbol	Meaning
P	Protein pocket together with its atom features and coordinates.
x_0	Clean coordinates of the object being generated: ligand coordinates in TargetDiff or polygon coordinates in PolyGen2D.
x_t	Noisy coordinates at diffusion step t .
v_t	Discrete ligand atom-type state at diffusion step t in TargetDiff.
t	Diffusion timestep.
$\beta_t, \alpha_t, \bar{\alpha}_t$	Noise variance, per-step signal retention, and cumulative signal retention.
$\mu_\theta(x_t, t)$	Reverse-step mean implied by the denoiser.
$h_i^{(\ell)}$	Hidden embedding of node i at message-passing layer ℓ .
$\hat{a}_\phi$	Surrogate-predicted binding-affinity score.
$g(x_t, t)$	Arbitrary differentiable guidance scoring function.
λ_t	Guidance strength, including any time-dependent schedule.

Table 2.1: Main notation used.

Detailed schedule constants are defined in Appendix A.1.

2.2 Drug Discovery Context

In the past, Drug discovery was a slow and expensive process. Researchers would identify a biological target, manually design or screen molecules against it, and then test those molecules over stages of computational and experimental evaluation [4]. Although vast libraries of known compounds exist, the nearly infinite combinatorial space of possible molecules meant that this process was constrained by how many molecules could realistically be tested [5].

Machine learning introduces the possibility of *generating* candidate molecules instead of only screening preexisting ones. A generative model can learn the distribution of realistic molecules and then sample new ones that match the patterns present in the training data [6, 7]. In target-aware drug discovery, the goal becomes to generate molecules that are not merely realistic, but are likely to bind to a specific protein pocket [1, 8].

The task of this thesis is to understand whether the sampling of diffusion models for molecular generation can be improved using guidance from an external scoring function [9]. That question is important because in a realistic drug-discovery pipeline, binding is only one of several relevant objectives [5]. Ideally, a useful generative model should be steerable toward additional properties without having to retrain the entire model from scratch [4].

2.3 p53 and the Y220C Mutation

Guardian p53 is a vital protein present in every human cell – its task to regulate the life and death of its host cell [2, 10]. A functioning p53 prevents mutated cells from developing into lethal tumors and stopping cancer before it can form [2]. However, if p53 itself becomes mutated and can no longer perform its crucial role, the cell (and thus the entire body) is left vulnerable [10]. A mutation in p53 has been found to contribute to over 50% of all known cancers, so one can imagine the allure of restoring the mutant guardian [2, 10].

This study will focus on the Y220C mutant of p53, but the method is relatively target-agnostic [10]. Y220C is chosen because the molecule PK11000 has been found to restore it, and while PK11000 is not viable as a therapeutic drug due to its toxicity to the human body, it served as a proof-of-concept that functional restoration of the mutant is achievable [2, 10]. The ultimate goal in this lab is a pipeline that generates molecules that bind to and restore the target [10]. However, there does not yet exist a restoration score that can be used directly inside the generative process, and thus this thesis studies guidance in the more tractable setting of binding-affinity optimization while treating restoration-oriented guidance as a future extension [2, 10].

2.4 Molecular Docking

To evaluate whether a generated molecule is likely to bind to a target pocket, we use molecular docking, a calculation that quickly estimates how favorably a ligand fits into a protein binding site. Typically, a docking program tries to minimize the potential energy of the system and maps the result to a scalar score, with a lower score implying better docking. Docking is useful because it is computationally cheap, but it is still an estimate [4]. More importantly, it is not differentiable, meaning we cannot calculate how to change the molecule in order to minimize the docking score [5]. The primary docking software we will use is the Python implementation of AutoDock Vina [3].

2.5 Molecular Representation

A molecule is naturally represented as a graph, with atoms forming the nodes of the graph and bonds forming the edges citesternlieb2022targetspecific, alakhdar2024diffusion.

In three-dimensional generative modeling, each node also carries continuous geometric information in the form of coordinates, paired with discrete information such as atom type [1, 7, 8].

Simple fixed-length vectors are not a sufficient representation, since molecules vary in size, have local structure, and contain meaning in the relationships between atoms [10]. Thus, a useful molecular generative model must utilize both the discrete and

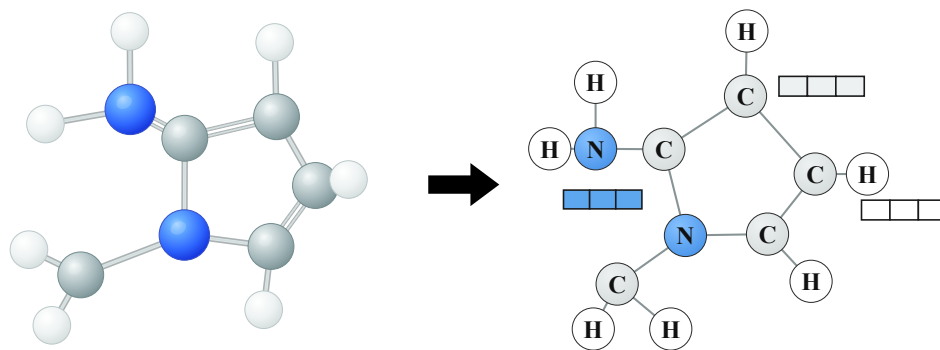


Figure 2.1: Molecule represented as a graph.

continuous information of the molecule while still remaining sensitive to the geometry of the system [4, 7, 8].

In our setting, the molecule’s quality depends on the protein pocket in which it is meant to bind, so the generative model must represent the protein-ligand complex in a way that allows the local and global context of the pocket to influence the molecule being generated [1, 4].

2.6 Deep Neural Networks

Neural networks take an input representation, transform it through a sequence of learned layers, and produce an output such as a denoised sample, a hidden embedding, or a scalar score [7, 10]. For our purposes, these are powerful tools that we can use as function approximators. A deep neural network is a parameterized function with enough flexibility to learn complicated mappings from data [11].

The basic computation in one layer is an affine transformation followed by a non-

linearity:

$$h^{(\ell+1)} = \sigma(W^{(\ell)}h^{(\ell)} + b^{(\ell)}) .$$

Here, $h^{(\ell)}$ is the representation at layer ℓ , $W^{(\ell)}$ and $b^{(\ell)}$ are the learned weights and biases, and σ is a nonlinear activation function such as ReLU or SiLU.

Stacking many such layers yields a deep model

$$\hat{y} = f_{\theta}(x),$$

where x is the input, $\hat{y}$ is the prediction, and θ denotes the full collection of learned parameters.

Training means choosing θ so that predictions match the desired targets on example data. In compact form, this is written as

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \ell(f_{\theta}(x_i), y_i) .$$

where (x_i, y_i) is the i th training example, ℓ is the error function for one example, and $\mathcal{L}$ is the average training loss that optimization tries to minimize.

In practice, the parameters are updated by an optimizer such as gradient descent or one of its variants. A basic gradient descent step is

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}(\theta) .$$

where η is the learning rate, which controls the step size, and $\nabla_{\theta} \mathcal{L}(\theta)$ is the gradient of the loss with respect to the parameters. In this thesis, these calculations are abstracted away in practice by PyTorch and its automatic differentiation tools [12]. However, the underlying idea is conceptually important: subtracting the gradient with respect to parameters moves them in a direction that decreases the loss, while adding the gradient with respect to some variable moves that variable in a direction that increases the quantity being differentiated. This same intuition later appears in guidance, where gradients are taken with respect to the current sample rather than with respect to the model parameters [13, 14].

Later models differ mainly in how they represent the input and how they structure the layers: a multilayer perceptron applies dense layers to vectors, a graph neural network adds message passing over nodes and edges, and a diffusion denoiser uses the same general training logic but with targets determined by the diffusion process [1, 6].

2.7 Graph Neural Networks

The fundamental challenge of machine learning is enabling the model to truly understand the meaning of the data it is intended to learn from. This problem is known as representation learning, and techniques are constructed based on the format of the data – convolutional neural networks to understand grid-like data, transformers to understand sequence data. In our case, we will use graph neural networks (GNNs),

which are designed for graph-structured data. In a graph, the meaning of a node depends on its own features as well as the nodes and edges surrounding it [15]. GNNs address this by using message passing, during which each node repeatedly aggregates information from neighboring nodes and updates its internal representation accordingly [16].

A message-passing update can be written as

$$m_i^{(\ell)} = \sum_{j \in \mathcal{N}(i)} \psi(h_i^{(\ell)}, h_j^{(\ell)}, e_{ij}), \quad h_i^{(\ell+1)} = \phi(h_i^{(\ell)}, m_i^{(\ell)}).$$

where $\mathcal{N}(i)$ is the neighborhood of node i , $m_i^{(\ell)}$ is the information gathered from its neighbors at layer ℓ , and ψ and ϕ are learned update functions [16].

In general, GNN prediction tasks are done at the node level, edge level, or graph level. For example, one might predict the type of a masked atom, the presence of a bond, or a graph-level property such as binding affinity. Because molecules vary in size and because their structure is inherently relational, GNNs are a much more natural fit than standard multilayer perceptrons in most realistic molecular settings [1, 15].

2.8 SE(3)-Equivariance

Molecules exist in three-dimensional space, and their physical meaning does not depend on an arbitrary choice of coordinate frame [1, 7]. If a protein-ligand complex

is rotated or translated in space, it is still the same complex [1, 8]. A target-aware molecular generative model should therefore behave consistently under such transformations [1, 4, 8].

This requirement is captured by the idea of SE(3)-equivariance. The Special Euclidean Group in 3-Dimensions (SE(3)) consists of three-dimensional rotations and translations [1, 4, 7, 8]. A model is equivariant to these transformations if transforming the input results in a corresponding transformation of the output instead of producing an unrelated change [1, 8]. This is desirable in our setting, because if the protein pocket is rotated, the generated ligand should rotate with it in a coherent way [1, 4, 8].

Mathematically, equivariance means the output transforms with the input.

$$f(Rx + r) = Rf(x) + r,$$

Here, x is the input geometry, f is the model, R is a rotation, and r is a translation [1, 4, 8].

In our case, SE(3)-equivariance is beneficial because the geometric relationship between the protein and the ligand is important, but the absolute world-frame orientation of the complex does not. A model that ignores this principle risks learning meaningless dependencies on arbitrary coordinate choices instead of pure chemistry [1, 4].

2.9 Pre-Diffusion Approaches

Previous methods used by the Thayer Lab to approach this task include actor-critic autoregressive methods, though none have yet attempted a model that generates molecules that both bind to and restore mutant p53. Theodore B. Sternlieb’s thesis used deep reinforcement learning to fine-tune a pretrained generative model over molecular graphs, biasing it with reward functions that encourage desirable molecular properties and high binding affinity to a target protein [10].

Actor-critic methods could be extended to restore mutant p53, as another reward can easily be wired into the RL feedback loop, and this would be a promising approach if the method itself were not intrinsically flawed. The key to generative AI and machine learning in general is properly representing and reflecting the structure and behavior of your data, and, in this case, generating a molecule atom-by-atom violates that by imposing an artificial ordering on the atoms of the molecule [1, 4, 8]. Although realistic molecules can be formed using this approach, in general they tend to be less realistic and more unstable [8]. Thus, current state-of-the-art models use a different approach entirely which eliminates this defiance of nature: diffusion [1, 4, 8].

2.10 Diffusion Models

Diffusion models are generative models that learn to reverse a gradual noising process [6, 7]. Starting from a clean datapoint, noise is repeatedly added until the datapoint is transformed into pure Gaussian noise [6, 7, 13]. A neural network is then trained to reverse that process step by step, turning noise back into a realistic sample [6, 7].

This makes diffusion an appealing framework for generation [6]. Since the model does not need to construct a sample all at once, during the early steps it can recover coarse structure, while later steps refine details, a behavior is very applicable domains like image generation and molecular generation [6].

Historically, diffusion can be connected to earlier latent-variable generative models such as variational autoencoders [6]. VAEs learn an encoder and decoder jointly, mapping data into a latent probability distribution and then reconstructing the input from that distribution [6]. Diffusion simplifies this by fixing the forward corruption process and learning only the reverse denoising process, leading to more stable generation and sharper samples [6, 13].

2.11 Denoising Diffusion Probabilistic Models

The diffusion framework most relevant to this thesis is the denoising diffusion probabilistic model (DDPM) [6, 7]. In a DDPM, a forward process gradually corrupts

a clean sample into noise according to a fixed schedule. The reverse model then learns to predict how to remove that noise one step at a time [6, 7].

A parameterization common in image generation implementations trains the model to predict the Gaussian noise that was added at a given timestep, called the ϵ -prediction parameterization [6, 13]. Another mathematically equivalent parameterization known as the mean-prediction (x_0 -prediction) predicts the denoised sample directly [6]. In our geometric setting, the mean-prediction is more effective, since it allows the model to focus on learning the overall structure of each clean data-point rather than the fine-grained adjustments needed in perfecting each pixel of a generated image [1, 4, 8].

The forward corruption step can be written in closed form as

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I),$$

when x_0 is the clean sample, x_t is its noisy version at step t , $\bar{\alpha}_t$ controls how much signal remains, and ϵ is Gaussian noise. The learned reverse step then samples from

$$x_{t-1} \sim \mathcal{N}(\mu_\theta(x_t, t), \sigma_t^2 I).$$

when $\mu_\theta(x_t, t)$ is the denoiser’s estimated step back toward the data distribution, and σ_t sets how much randomness is still injected [6, 7].

The two common prediction parameterizations are connected by

$$\hat{x}_{0,\theta}(x_t, t) = \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \hat{\epsilon}_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}}.$$

when $\hat{\epsilon}_\theta$ is the model’s predicted noise and $\hat{x}_{0,\theta}$ is the corresponding estimate of the original clean sample [6].

2.12 Guidance in Diffusion Models

An attractive feature of diffusion models is that their sampling process can be modified without retraining the base denoiser. In classifier guidance, for example, a separate model is trained to predict class labels from noisy inputs. During reverse diffusion, the gradient of that classifier with respect to the current sample is used to shift the reverse step toward higher probability under the target class [13, 14, 17].

In principle, any differentiable scoring model can be used to guide reverse diffusion in any domain. One can imagine steering samples toward higher affinity, higher synthesizability, or any other property that can be evaluated on the current noisy state [1, 4, 5, 9].

In that case, a generic guided reverse update takes the form

$$x_{t-1} \sim \mathcal{N}(\mu_\theta(x_t, t) + \lambda_t \nabla_{x_t} g_\phi(x_t, t), \sigma_t^2 I),$$

when g_ϕ is the external score, $\nabla_{x_t} g_\phi$ points in the direction that improves that score,

and λ_t controls how strongly guidance affects the step [5, 13, 17].

In this study, we explore the application of guidance in diffusion models in molecular generation.

3 Methods

3.1 Overview

This thesis studies guidance in two settings. We evaluate guidance directly in TargetDiff, a target-aware molecular diffusion model trained to generate molecules inside a protein pocket [1, 4]. We then introduce PolyGen2D, a controlled synthetic testbed designed to isolate the mechanics of guidance in a simpler coordinate-based diffusion setting. TargetDiff is the real target-aware molecular model of interest, but it contains many interacting sources of complexity [1, 8]. PolyGen2D removes many of those confounding factors so that specific questions about guidance can be studied more directly.

We first generate and evaluate molecules using TargetDiff, then train a surrogate affinity model and test its use for sampling-time guidance. In order to more reliably improve generation, we introduce PolyGen2D to study guidance in a controlled setting.

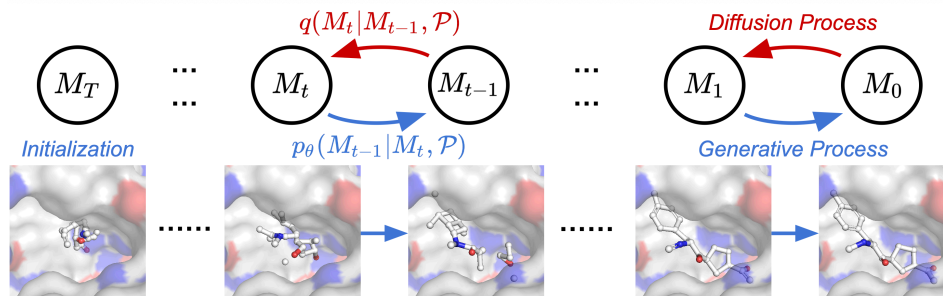


Figure 3.1: TargetDiff Diffusion Process [1]

3.2 TargetDiff Pipeline

3.2.1 Model Setup

TargetDiff is a diffusion model based on the DDPM framework that generates molecules in three dimensions conditioned on a protein pocket. It is trained on a dataset of millions of protein-ligand complexes and is designed to produce ligand structures that fit inside a target pocket while respecting the geometric structure of the protein-ligand environment.

For the purposes of this thesis, TargetDiff is treated as a frozen pretrained generative model. We do not retrain the model itself. Instead, we use it as the underlying denoiser and ask whether its sampling process can be improved by injecting an additional guidance signal derived from a separate scoring model. We ask whether a strong existing target-aware diffusion model can be steered at sampling time without changing its core generative backbone.

Our protein of interest is mutant p53 Y220C. Because p53 is not part of the original

training distribution in any direct supervised sense, this setting also tests whether the learned molecular prior in TargetDiff generalizes well enough that meaningful pocket-aware generation is possible for a new target of interest.

3.2.2 Candidate Generation and Evaluation

We first use raw TargetDiff to sample a large collection of candidate molecules for the Y220C pocket. These generated molecules are then postprocessed and evaluated. The primary quantitative evaluation is docking score, obtained using AutoDock Vina [3]. In addition to docking, we filter generated molecules using simple criteria such as QED and synthesizability to remove low-quality samples.

This stage serves as a baseline evaluation and as the source of labeled data for training the surrogate affinity predictor.

3.3 Surrogate Guidance (Molecular)

3.3.1 Dataset Construction

The surrogate dataset is constructed from TargetDiff-generated molecules labeled with docking scores. Training on model-generated samples ensures that the predictor is aligned with the distribution produced by the base model against our chosen pocket.

3.3.2 Surrogate Model

We designed the architecture of a surrogate affinity model as a graph neural network built over a protein-ligand graph containing ligand-ligand edges and ligand-protein edges. Node features consist primarily of atom types, while edges are constructed from spatial proximity and augmented with edge features such as edge type and distance. After message passing, ligand node representations are pooled and passed through an MLP to produce a scalar binding-affinity prediction.

The surrogate alternates message passing with a ligand-level readout:

$$h_i^{(\ell+1)} = \phi\left(h_i^{(\ell)}, \sum_{j \in \mathcal{N}(i)} \psi(h_i^{(\ell)}, h_j^{(\ell)}, e_{ij})\right), \quad \hat{a}_\phi = \text{MLP}\left(\frac{1}{|L|} \sum_{i \in L} h_i^{(L)}\right).$$

where $h_i^{(\ell)}$ is the hidden representation of atom i at layer ℓ , L is the set of ligand atoms, and $\hat{a}_\phi$ is the surrogate’s predicted affinity.

In our case, the choice of a GNN stems from the structure of the problem. Binding affinity depends on both the local geometry of the ligand and its interactions with the surrounding protein atoms. A graph model over the joint protein-ligand environment is therefore a natural way to represent the necessary information.

3.3.3 Guidance Mechanism

Once trained, the surrogate model is inserted into the reverse diffusion process as a guidance function [5, 13, 18]. During sampling, the current noisy ligand state x_t

is passed into the surrogate model, and the gradient of the predicted affinity with respect to the ligand coordinates is used to bias the reverse step [1, 5, 13]. The intended effect is that if the surrogate predicts that a small movement of the current sample would increase the final affinity score, then that direction should be added to the reverse denoising update [5, 9, 13].

Using that surrogate, the sampling-time perturbation can be written as

$$\Delta_t = \lambda_t \nabla_{x_t}(-\hat{a}_\phi(x_t, P)), \quad x_{t-1} = \mu_\theta(x_t, t, P) + \Delta_t + \sigma_t z, \quad z \sim \mathcal{N}(0, I).$$

Δ_t is the guidance shift added at time t , P denotes the protein pocket context, and z is fresh Gaussian noise used to keep sampling stochastic [1, 5, 6].

Conceptually, this is the same basic logic as classifier guidance in image diffusion. The diffusion model weights are left unchanged. Only the sampling process is modified, by adding a gradient term derived from an external differentiable model [4, 5, 13].

3.4 PolyGen2D Testbed

3.4.1 Motivation

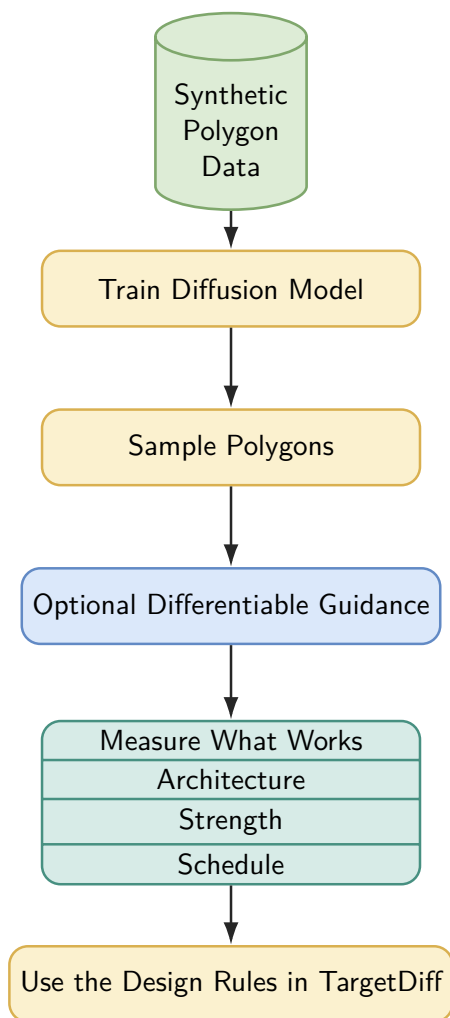


Figure 3.2: PolyGen2D-to-TargetDiff research workflow.

The molecular setting introduces multiple sources of complexity, including validity constraints, mixed discrete and continuous representations, and distribution shift across diffusion steps [1, 5, 7, 8]. These factors make it difficult to isolate the effect of guidance.

While our eventual goal is to integrate guidance into a TargetDiff-like model, we first build a working prototype and use it as a blueprint for the real version. PolyGen2D serves as a controlled synthetic testbed designed to isolate and evaluate guidance in coordinate-based diffusion models on graph-structured data.

3.4.2 Data and Representation

We construct synthetic polygon datasets, beginning with fixed-size near-regular hexagons and extending to variable-sized polygons with differing noise levels. The data distribution and target properties are fully controlled, and labels can be computed analytically.

3.4.3 Diffusion Models

The base PolyGen2D model is a classic DDPM over polygon coordinates. We begin with an MLP denoiser trained on a dataset of 100,000 near-regular hexagons. After training, we sample 10,000 hexagons from the frozen model, score both the training and sampled polygons using a regularity metric, and compare the resulting distributions. Ideally, the generated distribution should match the training distribution [6, 7, 13].

The synthetic diffusion model uses the same basic corruption and training template:

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \mathcal{L}(\theta) = \mathbb{E} \left[\|\hat{u}_\theta(x_t, t) - u\|_2^2 \right],$$

where x_0 is the clean polygon, x_t is its noisy version, and u is whichever target the denoiser is trained to predict, either the clean polygon itself (x_0 -prediction) or the added noise (ϵ -prediction) [6, 13].

Because the polygon data are graph-structured, we also implement GCN and GAT variants of the denoiser [19]. We also extend PolyGen2D to variable-size polygons.

3.4.4 Guidance Objectives

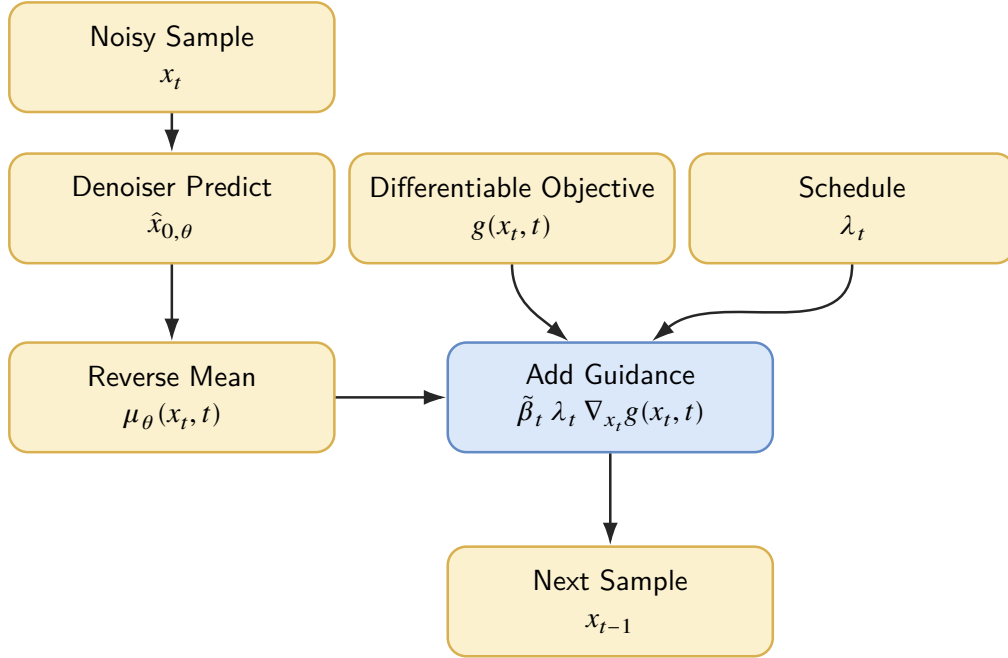


Figure 3.3: One guided reverse step in PolyGen2D

Guidance is implemented using differentiable polygon-level objectives. For the baseline PolyGen2D guidance experiments, the objective is a chosen score that can be evaluated on intermediate noisy states and differentiated with respect to the current polygon coordinates [13, 17].

Guidance modifies the reverse step as

$$x_{t-1} = \mu_{\theta}(x_t, t) + \tilde{\beta}_t \lambda_t \nabla_{x_t} g(x_t, t) + \sqrt{\tilde{\beta}_t} z, \quad z \sim \mathcal{N}(0, I).$$

When optimizing for more regular polygons, $g(x_t, t) = s_{reg}(x_t)$ is the current polygon’s regularity score, $\nabla_{x_t} s_{reg}(x_t)$ points toward more regular shapes, and $\tilde{\beta}_t$ scales the correction by the reverse-time noise level. In the case of optimizing pocket-conditioned experiments, PolyGen2D uses a differentiable pocket-fit score with $g(x_t, t) = s_{fit}(x_t)$ and $\nabla_{x_t} s_{fit}(x_t)$ pointing towards polygons that fit better. When using a neural network approximation, $g(x_t, t) = s_{fit_{\phi}}(x_t)$. This score acts as a geometric proxy for whether a polygon occupies a pocket by sampling points along the polygon boundary, rewarding samples that remain inside the pocket, and penalizing spillover, mismatched size, and unrealistic clearance. Appendix A.12–13 provides a precise definition for each of the guidance objectives.

3.4.5 Surrogate Variants

PolyGen2D also allows guidance experiments that would be difficult to run directly in the molecular setting. We compare surrogates trained on clean polygon states, surrogates trained on noisy polygon states, and surrogates trained on noisy states together with the timestep parameter. This allows us to directly compare the performance of these training regimes.

The noisy-state surrogate is trained by corrupting polygons and regressing the clean

regularity score:

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \mathcal{L}_{\text{reg}}(\phi) = \mathbb{E}_{x_0, t, \epsilon} \left[(\hat{y}_\phi(x_t, t) - s(x_0))^2 \right].$$

with $\hat{y}_\phi(x_t, t)$ the surrogate’s predicted score for a noisy polygon at time t , and $s(x_0)$ the clean polygon score it is trained to recover.

3.5 Experimental Design

3.5.1 TargetDiff Experiments

The molecular experiments begin with baseline generation using raw TargetDiff on the Y220C pocket [1, 2]. We then evaluate generated candidates with docking and filtering criteria and use that evaluated dataset to train the surrogate affinity predictor. Finally, we run guided sampling experiments in which the surrogate is inserted into reverse diffusion and the resulting samples are compared with the unguided baseline.

3.5.2 PolyGen2D Experiments

The synthetic experiments are designed to isolate specific questions that are hard to answer directly in the molecular setting. These include: how denoiser architecture changes the learned distribution, whether guidance changes the score distribution in the intended direction, whether clean-state surrogates differ from noisy-state surro-

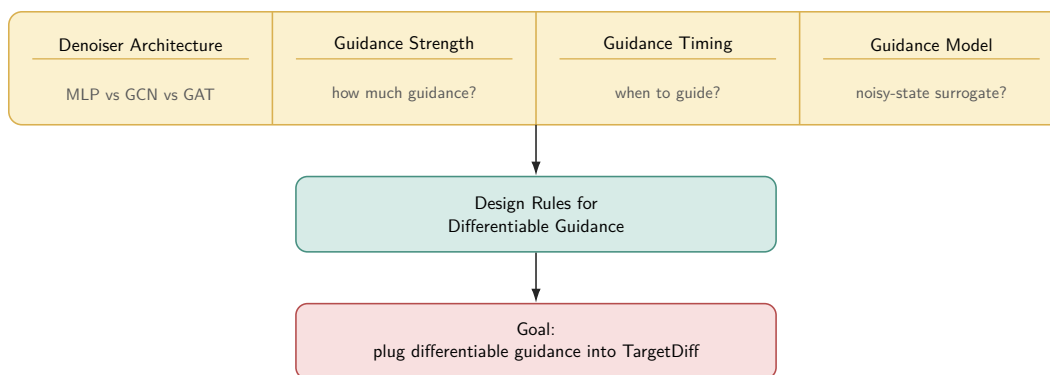


Figure 3.4: PolyGen2D methodology

gates, and whether including the timestep improves guidance quality.

3.5.3 Evaluation Metrics

In the molecular setting, the primary evaluation criteria are docking score, drug-likeness, synthesizability, and visual inspection of top-ranked candidates. In the polygon setting, the primary criteria are the visual quality of sampled polygons and the comparison between training and generated distributions.

4 Results

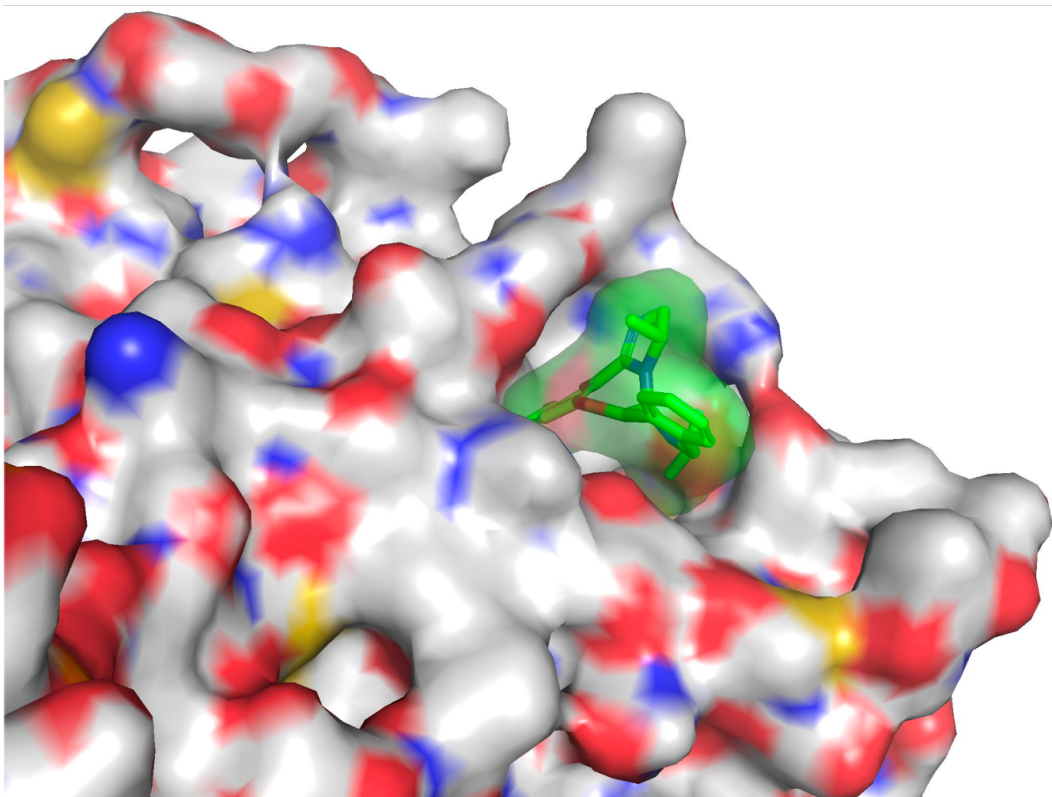


Figure 4.1: Docked generated molecule on Y220C

4.1 Baseline TargetDiff

The first stage of this work was to evaluate the baseline TargetDiff model on the Y220C pocket without any added guidance. Using the pretrained target-aware diffusion model, we generated a large collection of candidate molecules for the pocket of interest and then evaluated those candidates using molecular docking. This established a baseline for what the unguided model could already achieve before any attempt was made to steer sampling toward improved affinity.

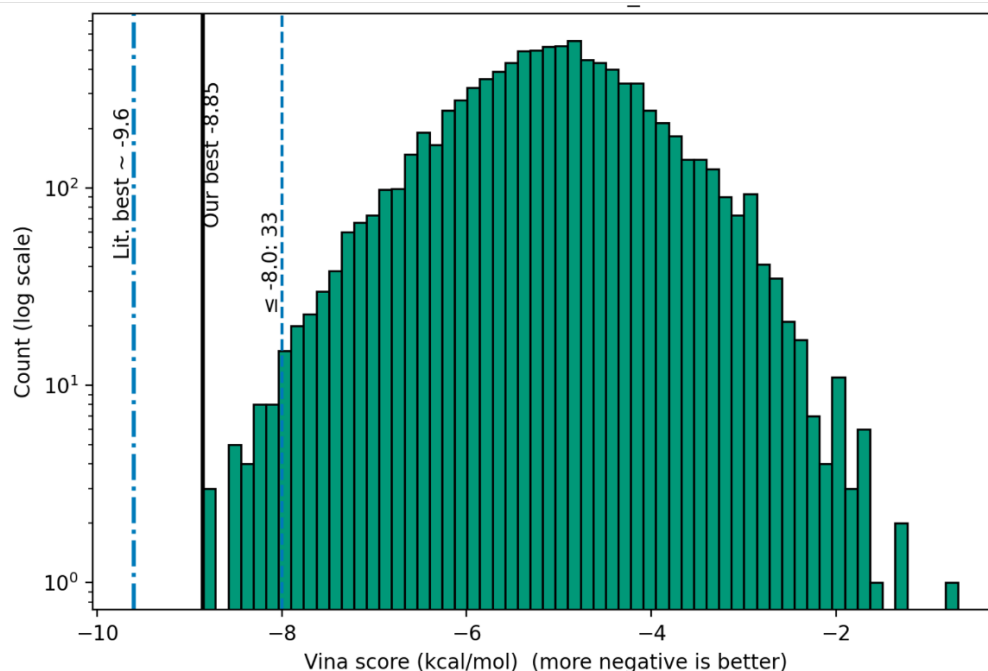


Figure 4.2: Score distribution of generated molecules

We successfully generated and scored over 10,000 molecules in our pocket of interest on Y220C. We filtered the molecules to accept minimum QED and synthesizability scores, then ranked them by binding affinity score and visualized the top candidates.

Figure 4.1 shows an example generated molecule docked in the Y220C pocket, while Figure 4.3 shows several of the top-ranked generated candidates.

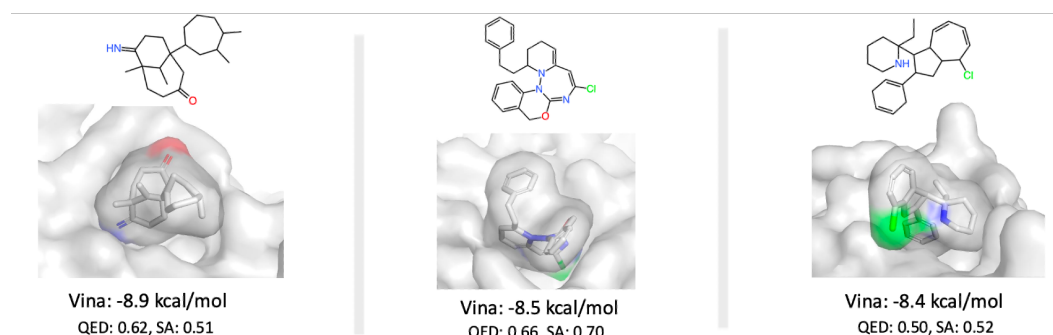


Figure 4.3: Top generated candidates

The results in Figure 4.2 show that raw TargetDiff is already capable of producing plausible target-aware candidates for Y220C. Molecules with a binding affinity score ≥ 8 kcal/mol are considered promising, but the best molecules in the literature still score better than the ones generated with raw TargetDiff.

4.2 Surrogate Predictive Performance

We trained a surrogate model on molecules generated by TargetDiff [1] and labeled by AutoDock Vina [3], and evaluated it on a held-out test set.

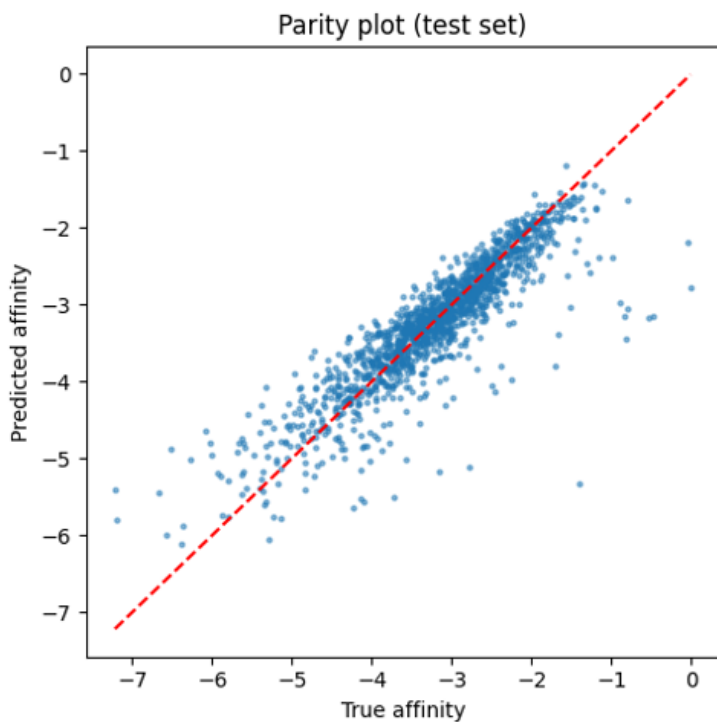


Figure 4.4: Surrogate affinity model standalone performance.

Although we evaluate the binding affinity of our final molecules using the Vina Dock score, which performs a minimization search to find the optimal pose of the molecule before scoring its affinity [3], we were careful to label it on the raw

score_only Vina label, since this allows the model to learn the prediction given its current 3D orientation in the pocket, not some other arbitrary orientation found by the Vina Dock algorithm. This allows the positional context that the GNN extracts to more effectively inform the model’s prediction.

The surrogate model learned to correctly predict the binding affinity of a molecule, as shown in the parity plot of the test set in Figure 4.4. On clean evaluated molecules, the model learned a relationship between molecular structure and docking score.

4.3 Guided PolyGen2D Performance

Type	Scale	Start	End	N	Score-only Δ	Dock Δ
σ	20	0.05	0.02	191	-0.013	-0.079
σ	10	0.05	0.02	191	-0.004	-0.077
fixed	0.2	0.05	0.02	191	-0.033	-0.069
σ	50	0.10	0.02	191	-0.018	-0.048
σ	8	0.10	0.02	191	-0.014	-0.045
fixed	0.2	0.10	0.02	190	0.011	-0.015
σ	10	0.10	0.02	191	-0.019	0.054
σ	20	0.10	0.02	191	-0.020	0.061
σ	8	0.05	0.02	191	-0.002	0.070
fixed	1.0	0.05	0.02	178	0.146	0.116
fixed	1.5	0.05	0.02	169	0.235	0.170
fixed	0.5	0.10	0.02	184	0.161	0.171
fixed	0.5	0.05	0.02	189	0.012	0.205
fixed	1.5	0.10	0.02	158	0.390	0.217
fixed	1.0	0.10	0.02	161	0.321	0.227
σ	50	0.05	0.02	191	-0.030	0.228

Table 4.1: Guided minus baseline.

Lower is better for Vina metrics [3]. Cell color encodes dock verdict: green working, yellow = mixed, red = not working.

We conducted a sweep across hyperparameters to try to isolate a mode for which the average docking score improved with the guidance model compared to the baseline model. Table 4.1 shows that when the surrogate was plugged into the reverse diffusion process under specific hyperparameters, it led to an improvement in the binding affinity of the generated molecules.

4.4 PolyGen2D Baseline Behavior

We began by training PolyGen2D on fixed-size near-regular hexagons. After training, we sampled a large set of polygons from the frozen model and compared the score distribution of the generated polygons with the score distribution of the training polygons using a regularity metric.

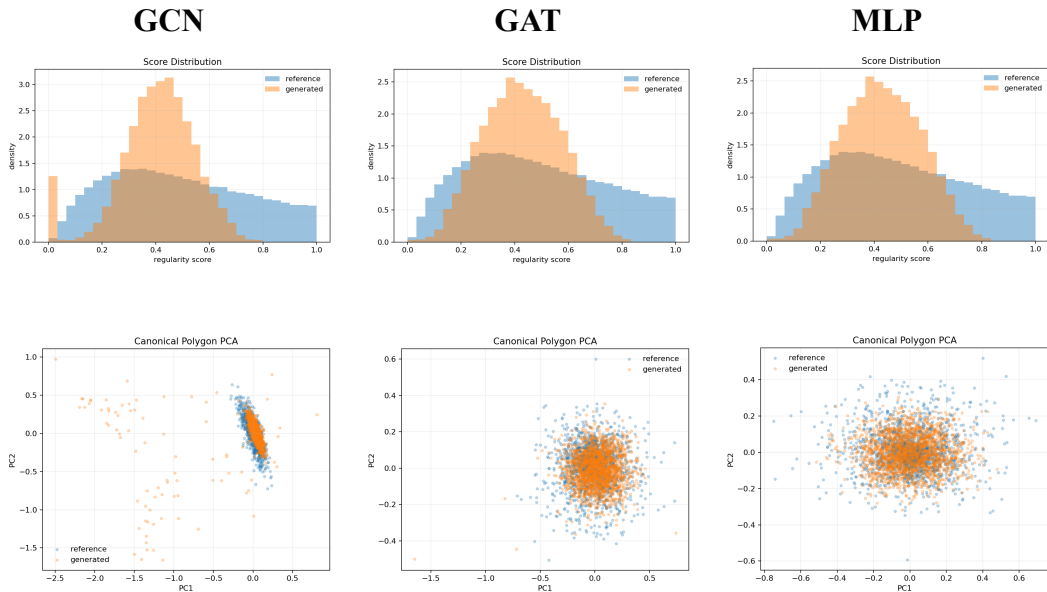


Figure 4.5: PolyGen2D unguided fixed-size results across architectures.

Before guidance, the model produced reasonable outputs, but the distribution of the

scored training data did not match the distribution of the scored sampled data, and some architectures matched the distribution better than others, as seen in Figure 4.5. The spike near 0.0 in the GCN score distribution is consistent with the many outliers shown on the PCA plot. The GAT has far fewer outliers, with the MLP having the least amount of outliers. See Appendix A.8 for details of the denoiser formulations.

4.5 Guidance in PolyGen2D

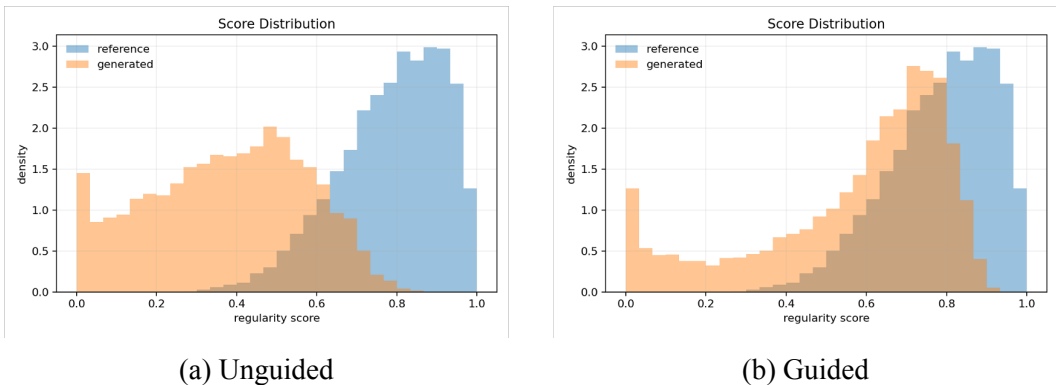


Figure 4.6: PolyGen2D unguided vs. guided (fixed-sized MLP)

We then added guidance in the form of a differentiable scoring function that calculates the regularity of a polygon. Figure 4.6 shows that with guidance enabled, the score distribution drastically shifted to favor higher-scoring polygons.

4.6 Surrogate Variants

In the case of predicting binding affinity in the molecular domain, a differentiable guidance function is intractable and a surrogate neural network approximator is instead required to use as a differentiable guidance function. Although our s_{fit} in

PolyGen2D is designed to be differentiable, we simulate the intractability by training a surrogate model s_{fit_ϕ} to predict polygons labeled by s_{fit} , and then compare the performance of a diffusion model guided by surrogates trained under different conditions.

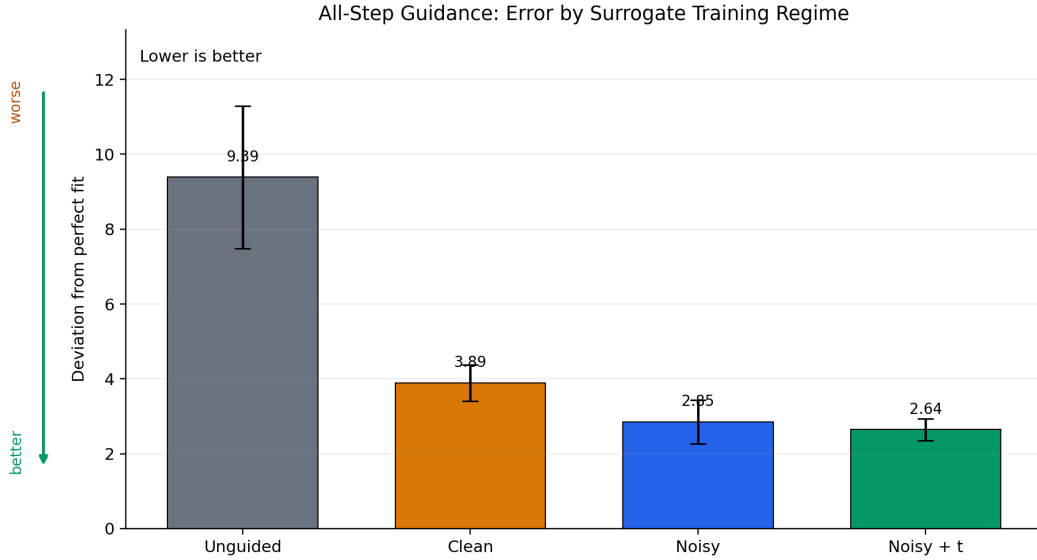


Figure 4.7: PolyGen2D Guided Sampling Performance

Figure 4.7 demonstrates that surrogates trained on noisy states outperform clean-state surrogates when used for guidance. Including timestep information as a feature of the neural network further improves performance.

4.7 Architectural Results

Figure 4.5 shows an unexpected architectural result produced by PolyGen2D. Because the synthetic data are graph-structured, we initially assumed that a graph neural network would be the natural and superior denoiser architecture. We therefore implemented both GCN and GAT variants in addition to the baseline MLP denoiser.

In the fixed-size regular-hexagon setting, however, performance actually decreased with the GCN, and the GAT was comparable but the PCA projection showed that it suffered from slightly more outliers compared to the MLP.

5 Discussion

5.1 Core Finding

The results support that yes, a diffusion model for molecular generation can be improved at sampling time by applying guidance from an external scoring function. In the molecular setting, guidance was capable of improving affinity under some hyperparameter settings, but those gains were modest and not robust across the full sweep. In the synthetic setting, by contrast, guidance produced much clearer improvements. Together, these results suggest that sampling-time guidance can work, but that its effectiveness depends strongly on how well the guidance signal matches the states encountered during reverse diffusion.

The molecular experiments show that a clean-state surrogate can produce a useful signal in some cases, but not a consistently strong one. The synthetic experiments show that when the guidance objective is trained with the data from within the noisy diffusion process in mind, the effect becomes much more pronounced.

5.2 Why Molecular Gains Remain Limited

In several settings, guided sampling improved average affinity relative to the baseline. However, these improvements were small and sensitive to hyperparameter choice, rather than broad and reliable.

Previous attempts were done inserting guidance early on in the diffusion process, which broke the model’s ability to bring molecules back to the realistic manifold and prevented it from generating valid molecules.

Since the guidance model requires relatively clean data as input to function at all, this means that guidance can only be inserted into the very end of the full diffusion process once the data it is seeing is clean enough, and act to make small improvements on the model instead of major structural improvements that it might otherwise be able to make early on.

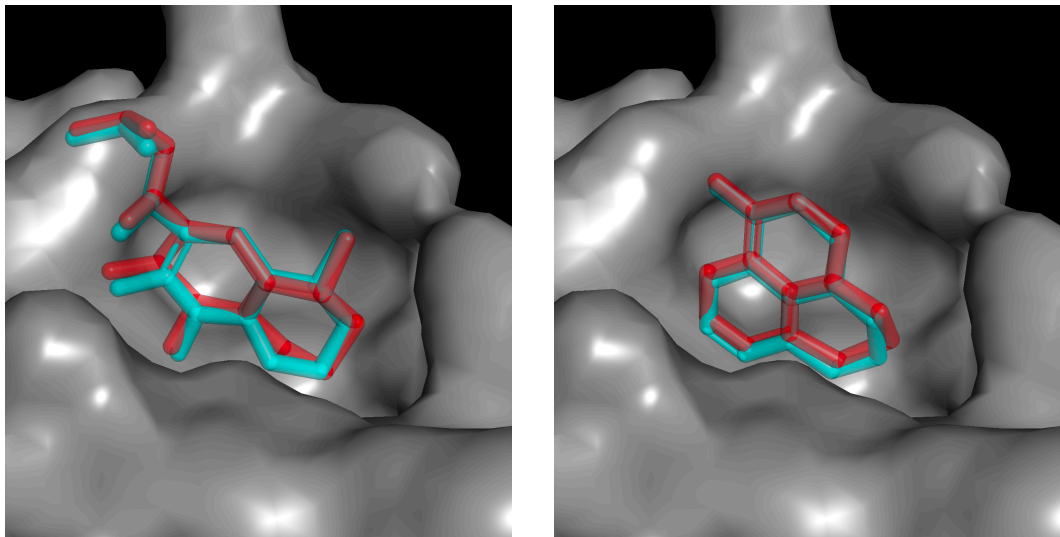


Figure 5.1: Overlay of guided and unguided generated molecules.

In Figure 5.1 the teal molecule is unguided, while the red molecule is guided.

A likely reason is a common ML failure mode: distribution mismatch. The surrogate is trained on clean molecular states, but during reverse diffusion it is evaluated on noisy intermediate states. Even if the surrogate predicts affinity reasonably well on completed molecules, its gradients become less useful when applied to partially

denoised samples that lie outside its training distribution. Thus, guidance may still push samples in a favorable direction, but the effect is too weak or unstable to produce a strong systematic gain.

5.3 What PolyGen2D Clarifies

In the molecular setting, many factors are entangled, e.g. chemical validity, mixed discrete and continuous structure, docking noise, and the mismatch between clean and noisy states. In the synthetic setting, we used PolyGen2D to removed or controll these, making it possible to test guidance more directly and interpret the results more easily.

In our synthetic environment, analytic guidance shifted the score distribution strongly in the intended direction, and surrogate-based guidance improved more effectively when the surrogate was trained on noisy states rather than only on clean ones. Including timestep information improved it further. These results support the idea that the main obstacle in the molecular setting is building a guidance model that is reliable across the diffusion trajectory.

5.4 Architectural Considerations

The PolyGen2D experiments also produced an architectural result worth interpreting carefully. In the fixed-size regular-hexagon setting, the MLP denoiser outperformed both GCN and GAT variants. This suggests that the denoising task in that

toy setting depends more on immediate access to the full global configuration than on local relational message passing.

That result should not be overgeneralized. Molecules are variable-size objects whose behavior depends heavily on local geometric interactions, so the case for graph-based and equivariant models in molecular generation remains strong. This tells us that the best denoiser depends on the structure of the task.

5.5 Implications for Molecular Guidance

In the future a stronger guidance model would likely need to be trained on noisy molecular states rather than only on clean states, and it would likely benefit from explicit timestep conditioning, thus optimized for the reverse-diffusion regime in which it will be used.

The practical difficulty is that noisy molecular states are much harder to label than clean ones. A distorted intermediate sample may no longer be chemically valid or even processable by docking software. This makes the molecular problem more challenging than the synthetic one.

In addition, our current surrogate model only moves the continuous coordinates of the molecule to optimize its objective and disregards the atom types. With only late-stage guidance on the coordinates, the model won't alter the distribution of molecules that the unguided denoiser would have otherwise created, since the generation trajectory won't be significantly altered by the guidance and will instead act

as a local coordinate refinement mechanism.

There is a justification for only optimizing coordinates – partly since an atom type vector represented using one-hot vectors is not differentiable. During each step of the diffusion sampling process, the model attempts to predict the final atom types v_0 given v_t , which it outputs as a probability distribution and then samples it into a categorical choice of atom type. If we use the smooth probability distribution as input to our affinity model, the model will likely suffer from the same distribution mismatch as it did in the case of receiving noisy coordinate inputs if it was trained on molecules with fixed atom types. If we soften the one-hot atom-type vectors, a small nudge in the vector will likely not affect the type of the atom. If the gradient was able to nudge it to flip an atom type, that may result in a drastic change in the prediction score. A potential future area of exploration is to compute the gradient w.r.t. the soft distribution, use that gradient to adjust the type logits after a prediction of the logits v_0 and before sampling, then sample from the adjusted categorical distribution. But this is still less principled than coordinate guidance since the gradient occurs in continuous space, after which a separate discontinuous action is performed.

Even so, these results indicate that guidance is worth pursuing, and identify and that a guidance model whose supervision matches the states visited during sampling is most effective.

6 Conclusion

6.1 What This Thesis Establishes

Molecular guidance produced modest, hyperparameter-sensitive gains, while guidance in PolyGen2D was much stronger and more reliable. This suggests that guidance quality depends on alignment between the guidance model and the noisy states seen during reverse diffusion.

6.2 What Remains Open

The long-term goal of this lab is to guide molecular generation towards restoration of mutant protein function. Although still out of our reach, this thesis brings us closer by providing a precise understanding of the difficulties of guidance in this setting and how to make it more likely to succeed. We hope that our results and methodologies will be applied in the future as this noble goal becomes more achievable.

Appendix A: Mathematical Details

A.1 Scope

This appendix collects the mathematical details most relevant to the thesis. For TargetDiff, the important formulas are the forward and reverse diffusion processes, the direct x_0 parameterization used for ligand coordinates, and the SE(3)-equivariant refinement update. For PolyGen2D, the important formulas are the DDPM setup, the configurable prediction target, the guided reverse mean, the differentiable polygon regularity objective, and the pocket-fit objective used in the synthetic pocket experiments.

The shared schedule quantities used throughout the appendix are

$$\alpha_t = 1 - \beta_t, \quad \bar{\alpha}_t = \prod_{s \leq t} \alpha_s, \quad \tilde{\beta}_t = \beta_t \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t}.$$

A.2 TargetDiff: Forward Diffusion

Let the protein pocket be

$$P = \{(x_i^P, v_i^P)\}_{i=1}^{N_P}, \quad x_i^P \in \mathbb{R}^3,$$

and let the ligand at diffusion step t be

$$M_t = [x_t, v_t], \quad x_t \in \mathbb{R}^{N_L \times 3}, \quad v_t \in \{0, 1\}^{N_L \times K}.$$

For ligand coordinates, the forward diffusion process is Gaussian:

$$q(x_t | x_{t-1}) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t^x} x_{t-1}, \beta_t^x I),$$

with closed-form marginal

$$q(x_t | x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t^x} x_0, (1 - \bar{\alpha}_t^x) I),$$

so that

$$x_t = \sqrt{\bar{\alpha}_t^x} x_0 + \sqrt{1 - \bar{\alpha}_t^x} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$

For discrete atom types, the diffusion process mixes the clean one-hot type with the uniform categorical distribution:

$$q(v_t | v_0) = \text{Cat}\left(v_t \left| \bar{\alpha}_t^v v_0 + \frac{1 - \bar{\alpha}_t^v}{K} \mathbf{1} \right.\right).$$

A.3 TargetDiff: Reverse Process and Direct x_0 Prediction

The reverse-time Gaussian posterior for coordinates is

$$q(x_{t-1} \mid x_t, x_0) = \mathcal{N}(x_{t-1}; \tilde{\mu}_t(x_t, x_0), \tilde{\beta}_t^x I),$$

with

$$\tilde{\mu}_t(x_t, x_0) = \frac{\beta_t^x \sqrt{\bar{\alpha}_{t-1}^x}}{1 - \bar{\alpha}_t^x} x_0 + \frac{\sqrt{\alpha_t^x} (1 - \bar{\alpha}_{t-1}^x)}{1 - \bar{\alpha}_t^x} x_t,$$

and

$$\tilde{\beta}_t^x = \beta_t^x \frac{1 - \bar{\alpha}_{t-1}^x}{1 - \bar{\alpha}_t^x}.$$

TargetDiff predicts the denoised ligand

$$[\hat{x}_0, \hat{v}_0] = \phi_\theta(M_t, t, P),$$

and then plugs those predictions into the closed-form posteriors:

$$\mu_\theta(M_t, t, P) = \tilde{\mu}_t(x_t, \hat{x}_0), \quad c_\theta(M_t, t, P) = \tilde{c}_t(v_t, \hat{v}_0).$$

For the version used in this thesis, the position branch uses the direct x_0 parameterization:

$$\hat{x}_0 = \phi_\theta^{(x)}(x_t, v_t, t, P).$$

The coordinate training loss is therefore a direct denoised-coordinate regression:

$$L_{\text{pos}} = \frac{1}{N_L} \sum_{i=1}^{N_L} \|\hat{x}_{0,i} - x_{0,i}\|_2^2.$$

A.4 TargetDiff: SE(3)-Equivariant Refinement

The key equivariant update alternates between hidden-state refinement and coordinate refinement:

$$h_i^{(\ell+1)} = h_i^{(\ell)} + \sum_{j \in \mathcal{V}, j \neq i} f_h(d_{ij}^{(\ell)}, h_i^{(\ell)}, h_j^{(\ell)}, e_{ij}),$$

$$x_i^{(\ell+1)} = x_i^{(\ell)} + \mathbf{1}_{\text{lig}}(i) \sum_{j \in \mathcal{V}, j \neq i} (x_i^{(\ell)} - x_j^{(\ell)}) f_x(d_{ij}^{(\ell)}, h_i^{(\ell+1)}, h_j^{(\ell+1)}, e_{ij}),$$

where

$$d_{ij}^{(\ell)} = \|x_i^{(\ell)} - x_j^{(\ell)}\|_2.$$

This allows TargetDiff to update geometry and node features while preserving SE(3)-equivariance.

A.5 PolyGen2D: Diffusion Schedule and Forward Process

For PolyGen2D, let the clean polygon be

$$x_0 \in \mathbb{R}^{2n}$$

in dense form, or equivalently

$$x_0 \in \mathbb{R}^{n \times 2}.$$

The diffusion schedule is

$$\beta_t = \beta_{\text{start}} + \frac{t}{T-1} (\beta_{\text{end}} - \beta_{\text{start}}), \quad \alpha_t = 1 - \beta_t, \quad \bar{\alpha}_t = \prod_{s=0}^t \alpha_s,$$

with posterior variance

$$\tilde{\beta}_t = \beta_t \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t}.$$

The forward corruption used by PolyGen2D is

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$

A.6 PolyGen2D: Prediction Target and Training Loss

PolyGen2D supports two prediction targets selected by the configuration field

`diffusion.prediction_target`.

In direct-clean-sample mode, the denoiser predicts

$$\hat{u}_\theta(x_t, t) = \hat{x}_{0,\theta}(x_t, t),$$

and the loss is

$$\mathcal{L}_{x_0}(\theta) = \mathbb{E}_{x_0, t, \epsilon} \left[\|\hat{x}_{0, \theta}(x_t, t) - x_0\|_2^2 \right].$$

In epsilon-parameterized mode, the denoiser predicts

$$\hat{u}_\theta(x_t, t) = \hat{\epsilon}_\theta(x_t, t),$$

and the loss is

$$\mathcal{L}_\epsilon(\theta) = \mathbb{E}_{x_0, t, \epsilon} \left[\|\hat{\epsilon}_\theta(x_t, t) - \epsilon\|_2^2 \right].$$

The two parameterizations are related by

$$\hat{x}_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \hat{\epsilon}_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}},$$

and equivalently

$$\hat{\epsilon}_\theta(x_t, t) = \frac{x_t - \sqrt{\bar{\alpha}_t} \hat{x}_{0, \theta}(x_t, t)}{\sqrt{1 - \bar{\alpha}_t}}.$$

Thus, even when trained in epsilon mode, the model can always be converted to an implied clean-sample prediction.

A.7 PolyGen2D: Reverse Mean and Sampling Rule

Reverse diffusion in PolyGen2D always computes the mean from a predicted clean sample. The posterior mean is

$$\mu_{\theta}(x_t, t) = \frac{\beta_t \sqrt{\bar{\alpha}_{t-1}}}{1 - \bar{\alpha}_t} \hat{x}_{0,\theta}(x_t, t) + \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} x_t.$$

If the model was trained in epsilon mode, the sampler first converts

$$\hat{\epsilon}_{\theta}(x_t, t) \rightarrow \hat{x}_{0,\theta}(x_t, t),$$

then applies the same posterior-mean formula. So both parameterizations share the same ancestral sampler after conversion to $\hat{x}_{0,\theta}$.

Without guidance, reverse sampling uses

$$x_{t-1} = \mu_{\theta}(x_t, t) + \sqrt{\tilde{\beta}_t} z, \quad z \sim \mathcal{N}(0, I).$$

A.8 PolyGen2D: Denoiser Forms

MLP denoiser. For the dense MLP denoiser, the flattened polygon is concatenated with its time embedding:

$$h^{(0)} = [x_t; \psi(t)].$$

Each hidden layer applies

$$h^{(\ell)} = \text{SiLU}(W^{(\ell)}h^{(\ell-1)} + b^{(\ell)}),$$

and the output layer predicts the configured diffusion target:

$$\hat{u}_{\theta}(x_t, t) = W_{\text{out}}h^{(L)} + b_{\text{out}}.$$

GCN denoiser. For the GCN denoiser, node features begin as

$$h_i^{(0)} = [v_i; \psi(t_g)],$$

where $v_i \in \mathbb{R}^2$ is the vertex coordinate and t_g is the timestep of the polygon containing node i .

The cycle adjacency is

$$A_{ij} = \begin{cases} \frac{1}{3}, & j \in \{i-1, i, i+1\}, \\ 0, & \text{otherwise.} \end{cases}$$

Each layer computes

$$\tilde{h}^{(\ell+1)} = Ah^{(\ell)}W^{(\ell)} + b^{(\ell)},$$

then applies a residual projection and nonlinearity:

$$h^{(\ell+1)} = \text{SiLU}(\tilde{h}^{(\ell+1)} + R^{(\ell)}h^{(\ell)}).$$

GAT denoiser. For the GAT denoiser, the node features are

$$h_i^{(0)} = [v_i; p_i; \psi(t_g)],$$

where p_i is the cycle positional encoding

$$p_i = \left[\sin\left(\frac{2\pi i}{n}\right), \cos\left(\frac{2\pi i}{n}\right), \sin\left(\frac{4\pi i}{n}\right), \cos\left(\frac{4\pi i}{n}\right) \right].$$

For head k , attention scores are computed as

$$e_{ij}^{(k)} = \text{LeakyReLU}(\langle a_{\text{src}}^{(k)}, \tilde{h}_i^{(k)} \rangle + \langle a_{\text{trg}}^{(k)}, \tilde{h}_j^{(k)} \rangle),$$

with normalized attention weights

$$\alpha_{ij}^{(k)} = \frac{\exp(e_{ij}^{(k)})}{\sum_{u \in \mathcal{N}(j)} \exp(e_{uj}^{(k)})}.$$

These architectures are used when comparing how MLP, GCN, and GAT denoisers behave on the polygon task.

A.9 PolyGen2D: Guidance Models

Guidance regressors are trained on noisy polygons. The noisy input is produced with the same forward diffusion equation:

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon.$$

For the clean-state surrogate ablation, this reduces to the $t = 0$ case: For comparison, the clean-only surrogate objective is

$$\mathcal{L}_{\text{reg}}^{\text{clean}} = (\hat{y}(x_0) - y)^2.$$

For regression, the target is the scalar regularity score,

$$y = s(x_0),$$

and the loss is

$$\mathcal{L}_{\text{reg}} = (\hat{y}(x_t, t) - y)^2.$$

A.10 PolyGen2D: Guided Reverse Mean

All guidance terms act on coordinates. If a guidance score returns $g(x_t, t)$, then the reverse mean becomes

$$\mu_{\theta}^{\text{guided}}(x_t, t) = \mu_{\theta}(x_t, t) + \tilde{\beta}_t \lambda_t \nabla_{x_t} g(x_t, t).$$

For regressor guidance,

$$g_{\text{reg}}(x_t, t) = \hat{y}(x_t, t).$$

For analytic regularity guidance,

$$g_{\text{regscore}}(x_t, t) = s(x_t),$$

or, with a target score s^* ,

$$g_{\text{regscore}}(x_t, t) = - (s(x_t) - s^*)^2.$$

If multiple guidance terms are active, they are summed:

$$g_{\text{total}}(x_t, t) = \sum_{k=1}^K g_k(x_t, t).$$

A.11 PolyGen2D: Guidance Scheduling

In scheduled guidance,

$$\lambda_t = \lambda w(t)$$

where λ is the overall strength, and $w(t)$ is the schedule shape.

This is used to control whether guidance should be applied at all steps or concentrated in later denoising stages.

A.12 Polygon Regularity Objective

Given polygon vertices $v_1, \dots, v_n \in \mathbb{R}^2$, define centered coordinates

$$\tilde{v}_i = v_i - \frac{1}{n} \sum_{j=1}^n v_j.$$

Let the RMS radius be

$$r_{\text{rms}} = \sqrt{\frac{1}{n} \sum_{i=1}^n \|\tilde{v}_i\|_2^2 + \varepsilon},$$

and normalized coordinates

$$\bar{v}_i = \frac{\tilde{v}_i}{r_{\text{rms}}}.$$

The edge lengths are

$$e_i = v_{i+1} - v_i, \quad \ell_i = \sqrt{\|e_i\|_2^2 + \varepsilon}.$$

Let θ_i denote the interior angles, and let the coefficient of variation be

$$\text{CV}(a_1, \dots, a_n) = \frac{\sigma_a}{\mu_a + \varepsilon},$$

where

$$\mu_a = \frac{1}{n} \sum_{i=1}^n a_i, \quad \sigma_a = \sqrt{\frac{1}{n} \sum_{i=1}^n (a_i - \mu_a)^2 + \varepsilon}.$$

The differentiable regularity score is then

$$\text{CV}_{\text{edge}} = \text{CV}(\ell_1, \dots, \ell_n), \quad \text{CV}_{\text{angle}} = \text{CV}(\theta_1, \dots, \theta_n), \quad \text{CV}_{\text{radius}} = \text{CV}(\|\bar{v}_1\|_2, \dots, \|\bar{v}_n\|_2),$$

and

$$s(v) = \exp(-\alpha \text{CV}_{\text{edge}} - \beta \text{CV}_{\text{angle}} - \gamma \text{CV}_{\text{radius}}).$$

This is the analytic objective used in the regularity-guidance experiments.

A.13 PolyGen2D Pocket-Fit Objective

For pocket-conditioned experiments, PolyGen2D uses a differentiable geometric fit score instead of an external docking procedure. Points are sampled along each polygon edge as

$$q_{i,f} = (1 - f) v_i + f v_{i+1}.$$

Here, $q_{i,f}$ is a sampled boundary point between consecutive vertices v_i and v_{i+1} , and the collection of all such samples is denoted by $\mathcal{Q}$.

To estimate how much of the polygon stays inside the pocket, the score computes

$$\text{inside_fraction} = \frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} \text{sigmoid}\left(\frac{\delta_q}{\tau_{\text{in}}}\right).$$

Here, δ_q is the signed pocket margin of sample point q , so positive values mean the point is inside the pocket, and τ_{in} controls how softly that inside/outside transition is measured.

The full pocket-fit reward is then

$$\begin{aligned} \text{fit_score}(x) = & \text{inside_fraction} - w_{\text{out}} \text{outside_penalty} \\ & - w_{\text{area}} \text{area_ratio_penalty} - w_{\text{clear}} \text{clearance_penalty}. \end{aligned}$$

Here, the outside penalty measures how far boundary samples extend beyond the pocket, the area-ratio penalty discourages polygons with the wrong overall size relative to the pocket, and the clearance penalty keeps the polygon from hugging the wall too tightly or floating too loosely inside the pocket.

Appendix B: PolyGen2D Output

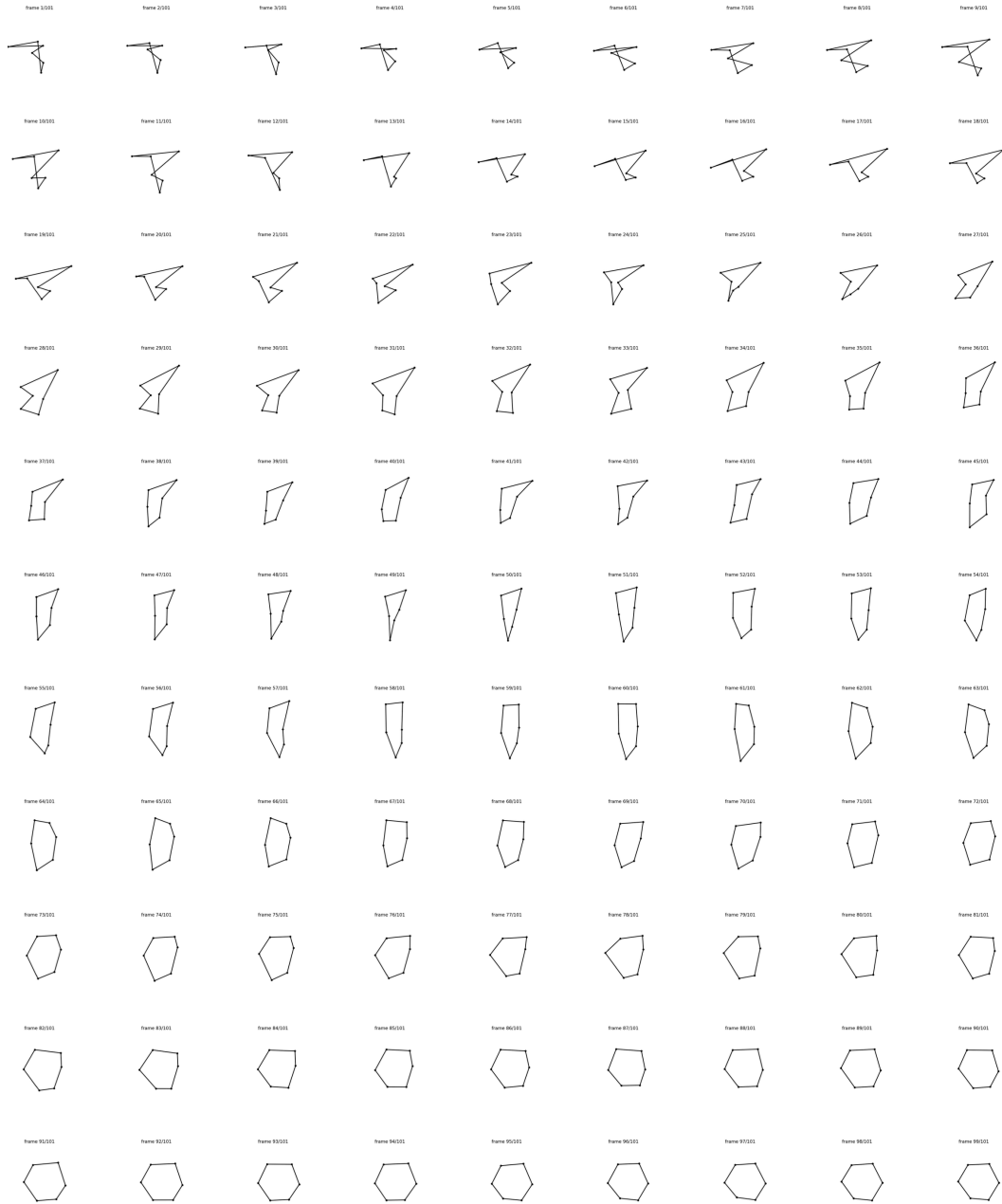


Figure 6.1: Trajectory of denoising a polygon.

Training aligned polygon sheet

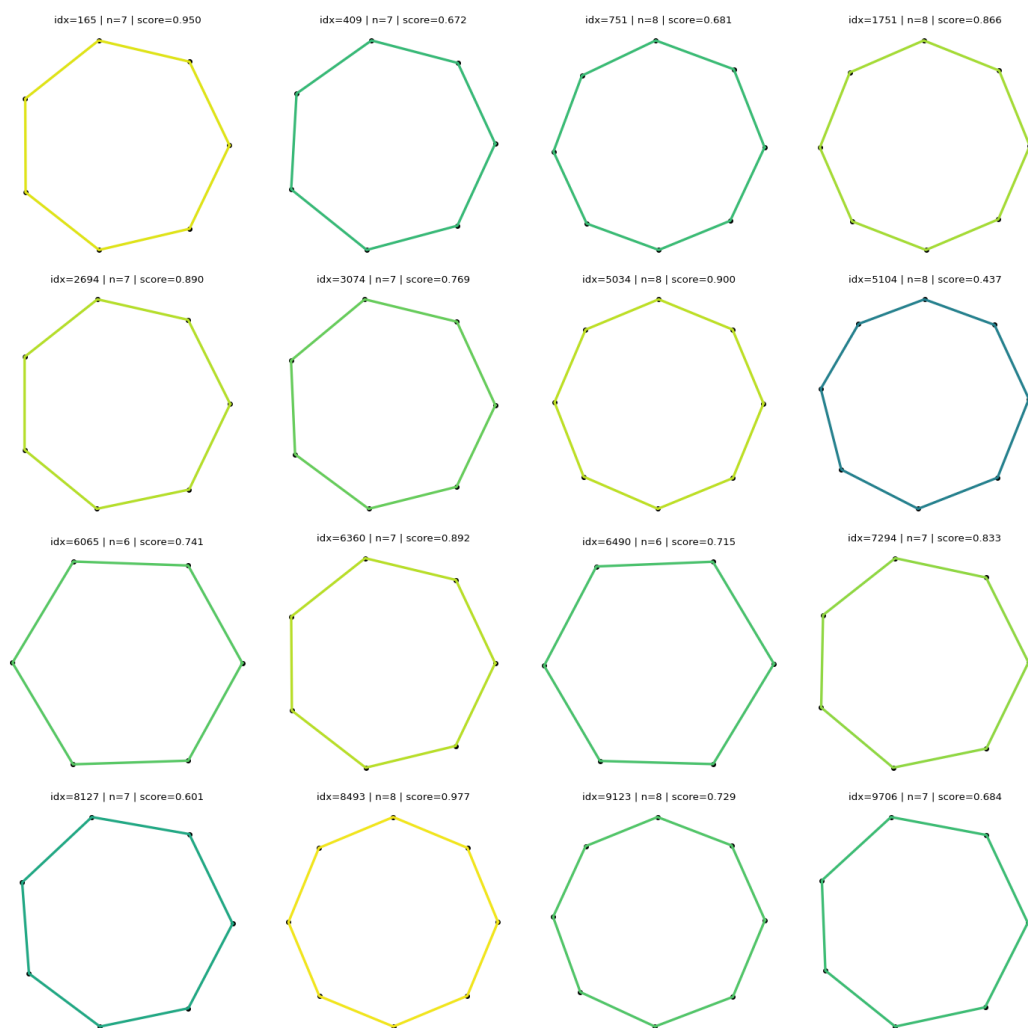


Figure 6.2: PolyGen2D training data.

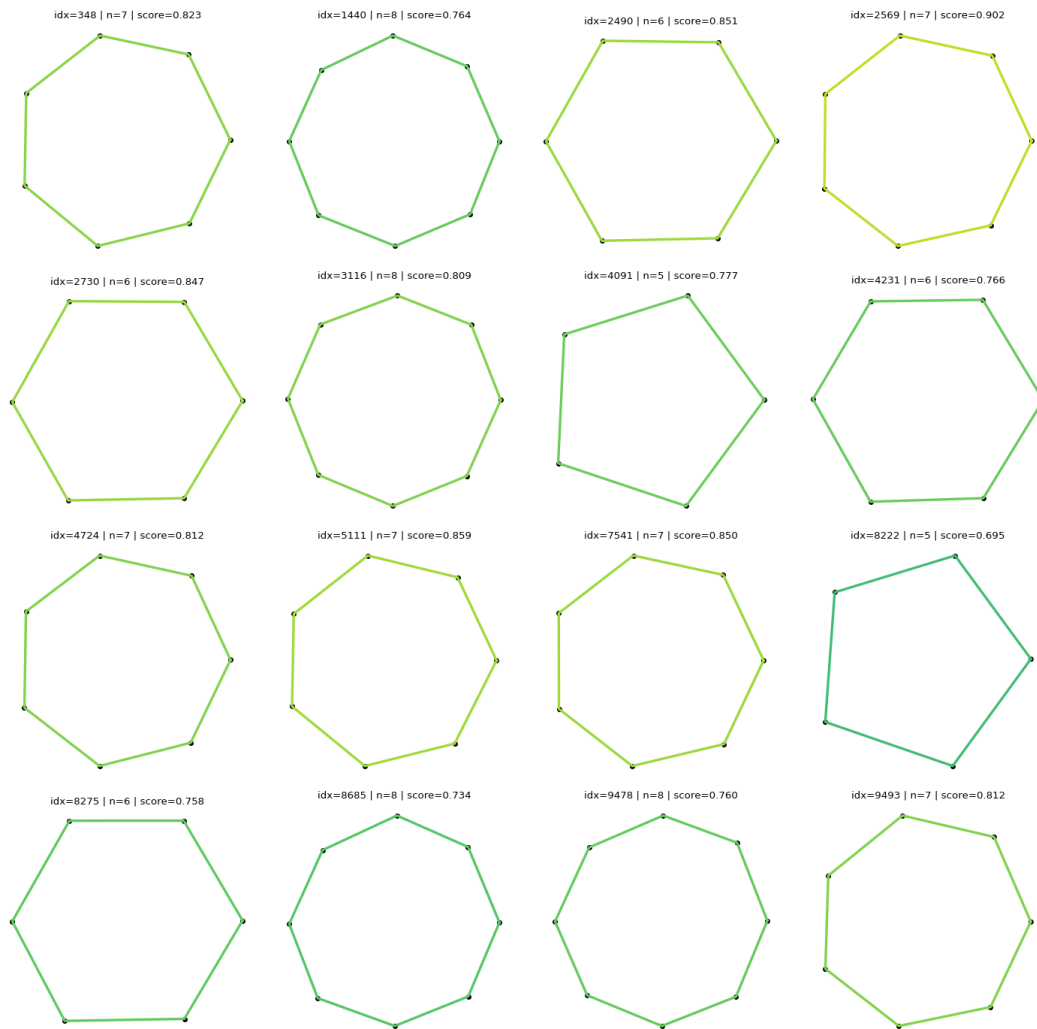


Figure 6.3: PolyGen2D generated data.

Appendix C: TargetDiff Output

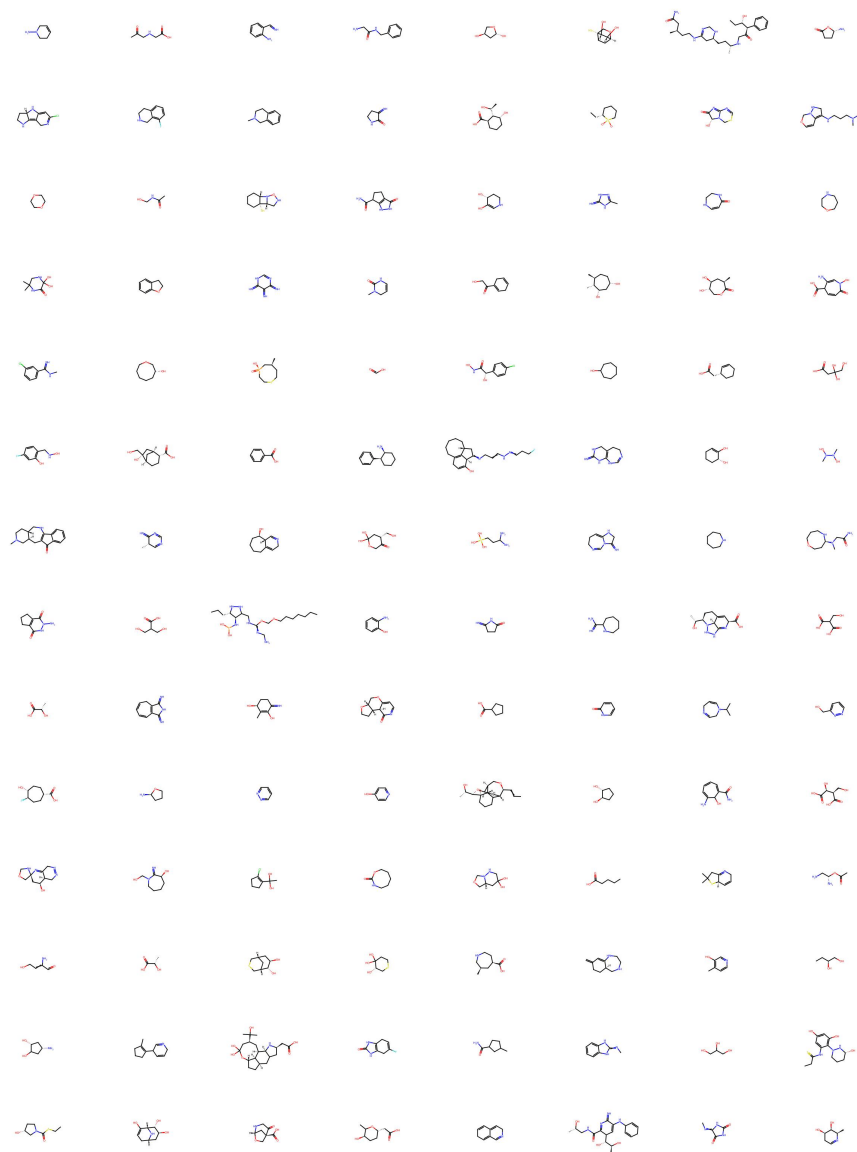


Figure 6.4: Molecules generated by TargetDiff.

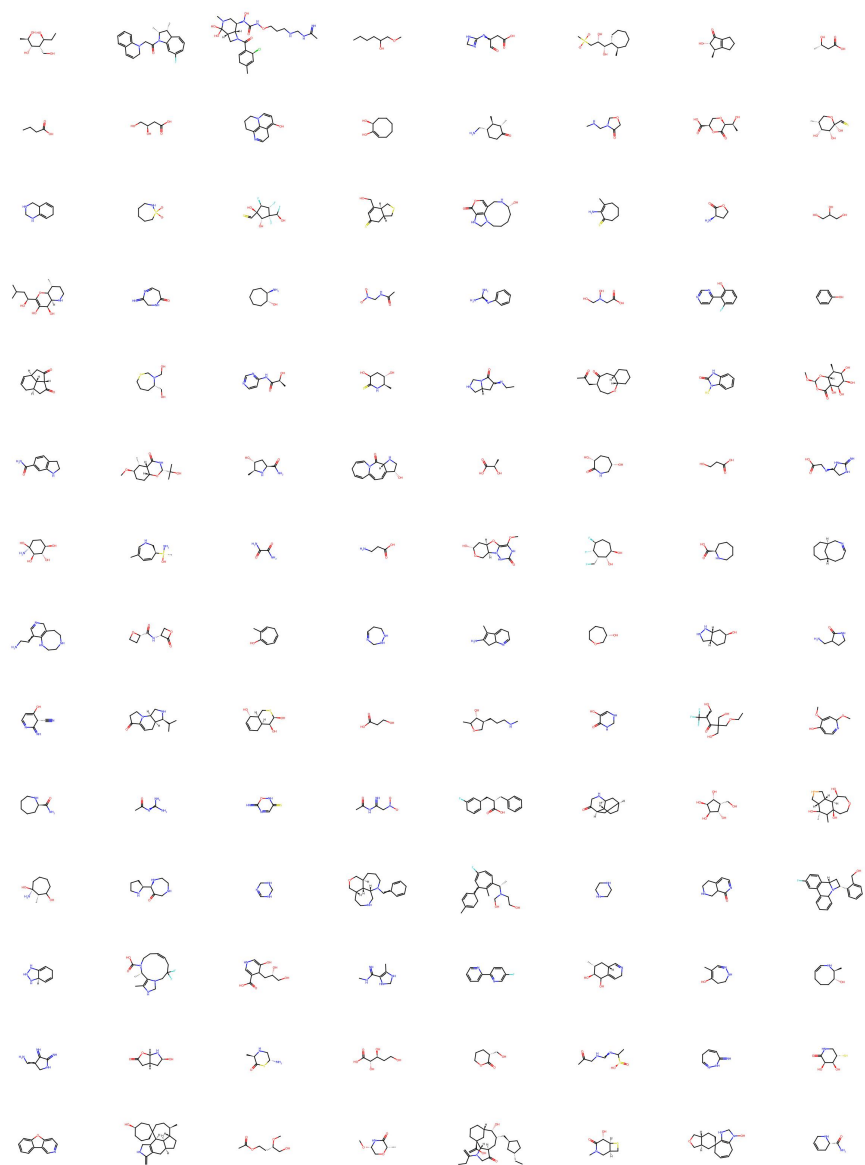


Figure 6.5: More molecules generated by TargetDiff.

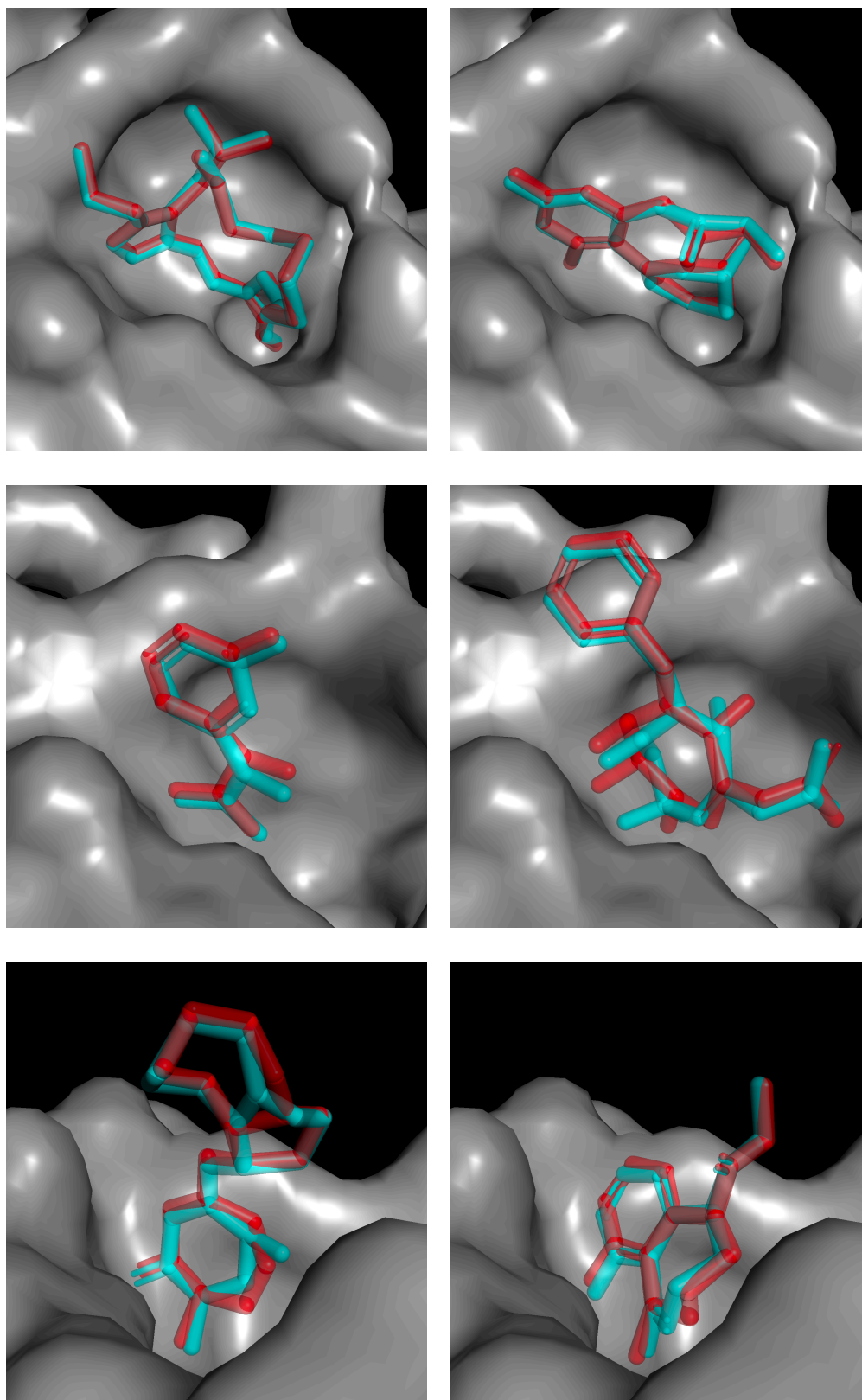


Figure 6.6: More overlaid guided and unguided molecules.

Appendix D: Scripts

Python Code

These snippets are copied directly from the implementation, with only surrounding unrelated lines omitted.

Sampler-side guidance update

Listing 1: Actual sampler-side guidance block from `molopt_score_model.py`

```
##### guidance #####

pos_model_mean = self.q_pos_posterior(x0=pos0_from_e, xt=ligand_pos,
    t=t, batch=batch_ligand)

pos_log_variance = extract(self.posterior_logvar, t, batch_ligand)

sigma = (0.5 * pos_log_variance).exp()

nonzero_mask = (1 - (t == 0).float())[batch_ligand].unsqueeze(-1)

guidance_active = guidance_model is not None and guidance_scale > 0

within_guidance_window = (
    (guidance_start_idx is None or i <= guidance_start_idx)
    and (guidance_end_idx is None or i >= guidance_end_idx)
)

if guidance_active and within_guidance_window:
    with torch.enable_grad():
```

```

        lig_pos_leaf = ligand_pos.detach().clone().requires_grad_(
True)

        score = guidance_model(

            ligand_pos=lig_pos_leaf,

            ligand_v=ligand_v,

            batch_ligand=batch_ligand,

            batch_protein=batch_protein,

            protein_pos=protein_pos,

        )

        score_mean = score.detach().mean().item()

        J = score.sum()

        grad = torch.autograd.grad(J, lig_pos_leaf)[0]

        grad_mean = scatter_mean(grad, batch_ligand, dim=0)

        grad = grad - grad_mean[batch_ligand]

var = sigma ** 2

if guidance_scale_mode == "var":

    guidance_delta = guidance_scale * var * grad

elif guidance_scale_mode == "sigma":

    guidance_delta = guidance_scale * sigma * grad

elif guidance_scale_mode == "fixed":

    guidance_delta = guidance_scale * grad

else:

```

```

        raise ValueError(f"Unknown guidance_scale_mode {
guidance_scale_mode}")

    noise = torch.randn_like(ligand_pos)

    noise_term = nonzero_mask * (sigma * noise_scale) * noise

    ligand_pos_next = pos_model_mean + guidance_delta + noise_term
else:

    noise = torch.randn_like(ligand_pos)

    ligand_pos_next = pos_model_mean + nonzero_mask * (sigma *
noise_scale) * noise

##### guidance #####

```

Default wrapper forward pass

Listing 2: Actual GuidedLigandContextWrapper.forward code from guidance.py

```

def forward(self, ligand_pos, ligand_v, batch_ligand, batch_protein,
protein_pos=None):

    device = ligand_pos.device

    num_graphs = batch_ligand.max().item() + 1

    atomic_numbers = trans.get_atomic_number_from_index(

        ligand_v.detach().cpu(), mode=self.ligand_atom_mode

    )

```

```

z_lig = torch.tensor(atomic_numbers, dtype=torch.long, device=
device)

use_provided_protein = (
    protein_pos is not None
    and batch_protein is not None
    and protein_pos.size(0) == self.num_pocket_atoms * num_graphs
)

if use_provided_protein:
    pocket_pos = protein_pos
    pocket_batch = batch_protein
else:
    pocket_pos = self.pocket_pos_centered.repeat(num_graphs, 1)
    pocket_batch = torch.arange(num_graphs, device=device).
repeat_interleave(self.num_pocket_atoms)

z_pocket = self.pocket_z.repeat(num_graphs).to(device)

pos = torch.cat([ligand_pos, pocket_pos], dim=0)
z = torch.cat([z_lig, z_pocket], dim=0)
batch = torch.cat([batch_ligand, pocket_batch], dim=0)
node_type = torch.cat([
    torch.ones(ligand_pos.size(0), dtype=torch.long, device=
device),

```

```

        torch.zeros(pocket_pos.size(0), dtype=torch.long, device=
device),
    ])

    edge_index, edge_type = build_context_edges(

        pos=pos,

        node_type=node_type,

        batch=batch,

        r_ligand=self.r_ligand,

        r_cross=self.r_cross,

    )

    data = Data(

        pos=pos,

        z=z,

        batch=batch,

        node_type=node_type,

        edge_index=edge_index,

        edge_type=edge_type,

    )

    affinity = self.affinity_model(data)

    return -affinity

```

Directed context-edge construction

Listing 3: Build context edges code from guidance.py

```
def build_context_edges(pos, node_type, batch=None, r_ligand=5.0,
                        r_cross=6.0):
    if batch is None:
        batch = torch.zeros(pos.size(0), dtype=torch.long, device=pos
                             .device)

    edge_src = []
    edge_dst = []
    edge_type = []

    for b in batch.unique():
        mask_b = batch == b

        idx = mask_b.nonzero(as_tuple=True)[0]

        if idx.numel() == 0:
            continue

        sub_pos = pos[idx]
        sub_type = node_type[idx]

        lig_local = (sub_type == LIGAND_NODE).nonzero(as_tuple=True)
[0]

        poc_local = (sub_type == POCKET_NODE).nonzero(as_tuple=True)
[0]
```

```

if lig_local.numel() > 0:

    lig_pos = sub_pos[lig_local]

    dist_ll = torch.cdist(lig_pos, lig_pos, p=2)

    keep_ll = dist_ll <= r_ligand

    keep_ll = keep_ll & ~torch.eye(lig_local.numel(), dtype=
torch.bool, device=pos.device)

    src_ll, dst_ll = keep_ll.nonzero(as_tuple=True)

    if src_ll.numel() > 0:

        edge_src.append(idx[lig_local[src_ll]])

        edge_dst.append(idx[lig_local[dst_ll]])

        edge_type.append(torch.full((src_ll.numel(),),
EDGE_LIGAND_LIGAND, dtype=torch.long, device=pos.device))

if lig_local.numel() > 0 and poc_local.numel() > 0:

    lig_pos = sub_pos[lig_local]

    poc_pos = sub_pos[poc_local]

    dist_pl = torch.cdist(poc_pos, lig_pos, p=2)

    keep_pl = dist_pl <= r_cross

    src_pl, dst_pl = keep_pl.nonzero(as_tuple=True)

    if src_pl.numel() > 0:

        edge_src.append(idx[poc_local[src_pl]])

        edge_dst.append(idx[lig_local[dst_pl]])

```

```

        edge_type.append(torch.full((src_pl.numel(),),
EDGE_POCKET_TO_LIGAND, dtype=torch.long, device=pos.device))

if len(edge_src) == 0:

    return (

        torch.empty(2, 0, dtype=torch.long, device=pos.device),

        torch.empty(0, dtype=torch.long, device=pos.device),

    )

edge_index = torch.stack([torch.cat(edge_src), torch.cat(edge_dst
)], dim=0)

edge_type = torch.cat(edge_type, dim=0)

return edge_index, edge_type

```

PolyGen2D: Minimal Source Snippets

These snippets show the implementation of the training loss and guidance mean shift.

Listing 4: Core DDPM training loss in polygen2d/models/diffusion.py

```

t = torch.randint(0, self.config.n_steps, (b,)), device=self.device,
dtype=torch.long)

noise = torch.randn_like(x0)

x_t = self.q_sample(x0, t, noise, graph_batch=graph_batch)

```

```

model_output = self.predict_model_output(x_t, t, graph_batch=
    graph_batch)

if self.config.prediction_target == "epsilon":
    loss = nn.functional.mse_loss(model_output, noise)
else:
    loss = nn.functional.mse_loss(model_output, x0)

```

Listing 5: Guided reverse mean shift in `polygen2d/models/diffusion.py`

```

model_output = self.predict_model_output(x_t, t, graph_batch=
    graph_batch)

x0_pred = self._model_output_to_x0(x_t, t, model_output, graph_batch=
    graph_batch)

model_mean = self.q_posterior_mean(x0_pred, x_t, t, graph_batch=
    graph_batch)

if guidance_grad is not None:
    grad = guidance_grad(x_t, t) if graph_batch is None else
        guidance_grad(x_t, t, graph_batch=graph_batch)

    model_mean = model_mean + posterior_var_t * grad

```

References

- [1] Jiaqi Guan, Wesley Wei Qian, Xingang Peng, Yufeng Su, Jian Peng, and Jianzhu Ma. 3d equivariant diffusion for target-aware molecule generation and affinity prediction. In *International Conference on Learning Representations*, 2023.
- [2] In Sub M. Han and Kelly M. Thayer. Reconnaissance of allostery via the restoration of native p53 dna-binding domain dynamics in Y220C mutant p53 tumor suppressor protein. *ACS Omega*, 9:19837–19847, 2024. doi: 10.1021/acsomega.3c08509.
- [3] O. Trott and A. J. Olson. Autodock vina: Improving the speed and accuracy of docking with a new scoring function, efficient optimization, and multithreading. *Journal of Computational Chemistry*, 31(2):455–461, 2010. doi: 10.1002/jcc.21334.
- [4] Arne Schneuing, Charles Harris, Yuanqi Du, Kieran Didi, Arian Jamasb, Ilia Igashov, Weitao Du, Carla Gomes, Tom L. Blundell, Pietro Lio, Max Welling, Michael Bronstein, and Bruno Correia. Structure-based drug design with equivariant diffusion models. *Nature Computational Science*, 4:899–909, 2024. doi: 10.1038/s43588-024-00737-x.
- [5] Seul Lee, Jaehyeong Jo, and Sung Ju Hwang. Exploring chemical space with

- score-based out-of-distribution generation. In *Proceedings of the 40th International Conference on Machine Learning*, 2023.
- [6] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems*, 2020.
- [7] Amira Alakhdar, Barnabas Póczos, and Newell Washburn. Diffusion models in de novo drug design. *Journal of Chemical Information and Modeling*, 64: 7238–7256, 2024. doi: 10.1021/acs.jcim.4c01107.
- [8] Haitao Lin, Yufei Huang, Odin Zhang, Siqi Ma, Meng Liu, Xuanjing Li, Lirong Wu, Jishui Wang, Tingjun Hou, and Stan Z. Li. DiffBP: generative diffusion of 3d molecules for target protein binding. *Chemical Science*, 16: 1417–1431, 2025. doi: 10.1039/D4SC05894A.
- [9] Hao Qian, Wenjing Huang, Shikui Tu, and Lei Xu. KGDiff: towards explainable target-aware molecule generation with knowledge guidance. *Briefings in Bioinformatics*, 25(1):bbad435, 2024. doi: 10.1093/bib/bbad435.
- [10] Theodore Beck Sternlieb. Target specific drug design with deep reinforcement learning. Master’s thesis, Wesleyan University, April 2022.
- [11] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.
- [12] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca

- Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*, 2019. doi: 10.48550/arXiv.1912.01703.
- [13] Prafulla Dhariwal and Alex Nichol. Diffusion models beat GANs on image synthesis. In *Advances in Neural Information Processing Systems*, 2021.
- [14] Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance, 2022.
- [15] G. Gkarpounis, C. Vranis, N. Vretos, and P. Daras. Survey on graph neural networks. *IEEE Access*, 12:128816–128832, 2024. doi: 10.1109/ACCESS.2024.3456913.
- [16] Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. Neural message passing for quantum chemistry. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 1263–1272. PMLR, 2017.
- [17] Chieh-Hsin Lai, Yang Song, Dongjun Kim, Yuki Mitsufuji, and Stefano Ermon. The principles of diffusion models, 2025.
- [18] Tomer Weiss, Eduardo Mayo Yanes, Sabyasachi Chakraborty, Luca Cosmo, Alex M. Bronstein, and Renana Gershoni-Poranne. Guided diffusion for inverse molecular design. *Nature Computational Science*, 3(10):873–882, 2023. doi: 10.1038/s43588-023-00532-0.

- [19] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations*, 2017.